

“Año de la Esperanza y el Fortalecimiento de la Democracia”

UNIVERSIDAD PERUANA LOS ANDES



FACULTAD INGENIERIA
ESPECIALIDAD: SISTEMAS Y COMPUTACION

Semana 13 :Desarrollo Ejercicios

CURSO:

- Base de Datos II

DOCENTE:

- Fernández Bejarano Raúl Enrique

ESTUDIANTE:

- Rosales Crispín Aristóteles Onassis

Ciclo:

- 5°SEMESTRE

HUANCAYO- JUNIN
2026

TEMA 1. ANÁLISIS DE RENDIMIENTO CON SQL PROFILER Y EXTENDED EVENTS

Proyecto 1: Respaldo Completo Institucional de la Base de Datos GradosTitulos

Enunciado: La Oficina de Tecnologías de Información de la universidad ha detectado que el módulo de **registro de expedientes de bachiller y título profesional** presenta lentitud durante las horas de mayor demanda.

Codigo:

```
-- =====
-- PROYECTO 1: Monitoreo de consultas lentas (> 1000 ms)
-- Base de datos: GradosTitulos
-- Esquemas: Academico, Tramite, Evaluacion, Documento
-- =====

USE master;
GO

-- 1. Crear la sesión de Extended Events
IF EXISTS (SELECT * FROM sys.server_event_sessions WHERE name =
'MonitorConsultasLentas_GradosTitulos')
    DROP EVENT SESSION [MonitorConsultasLentas_GradosTitulos] ON SERVER;
GO

CREATE EVENT SESSION [MonitorConsultasLentas_GradosTitulos] ON SERVER
ADD EVENT sqlserver.sql_batch_completed
(
    ACTION (sqlserver.sql_text, sqlserver.database_id, sqlserver.username,
sqlserver.session_id)
    WHERE (
        [duration] > 1000000 -- 1000 ms en microsegundos
        AND [database_id] = DB_ID('GradosTitulos')
        AND (
            [sqlserver].[sql_text] LIKE N'%Academico%'
            OR [sqlserver].[sql_text] LIKE N'%Tramite%'
            OR [sqlserver].[sql_text] LIKE N'%Evaluacion%'
            OR [sqlserver].[sql_text] LIKE N'%Documento%'
        )
    )
)
ADD EVENT sqlserver.rpc_completed
(
    ACTION (sqlserver.sql_text, sqlserver.database_id, sqlserver.username,
sqlserver.session_id)
    WHERE (
        [duration] > 1000000
        AND [database_id] = DB_ID('GradosTitulos')
        AND (
            [sqlserver].[sql_text] LIKE N'%Academico%'
            OR [sqlserver].[sql_text] LIKE N'%Tramite%'
            OR [sqlserver].[sql_text] LIKE N'%Evaluacion%'
            OR [sqlserver].[sql_text] LIKE N'%Documento%'
        )
    )
)
ADD TARGET package0.event_file
(
    SET filename = N'D:\BaseDatos2026\MonitorConsultasLentas_GradosTitulos.xel',
    max_file_size = 50, -- MB
    max_rollover_files = 5
)
```

```

)
WITH (
    MAX_MEMORY = 4096 KB,
    EVENT_RETENTION_MODE = ALLOW_SINGLE_EVENT_LOSS,
    MAX_DISPATCH_LATENCY = 30 SECONDS,
    MAX_EVENT_SIZE = 0 KB,
    MEMORY_PARTITION_MODE = NONE,
    TRACK_CAUSALITY = OFF,
    STARTUP_STATE = OFF
);
GO

-- 2. Iniciar la sesión
ALTER EVENT SESSION [MonitorConsultasLentas_GradosTitulos] ON SERVER STATE =
START;
GO

-- 3. Verificar estado
SELECT name, state_desc FROM sys.server_event_sessions WHERE name =
'MonitorConsultasLentas_GradosTitulos';
GO

-- 4. Consultar eventos capturados (para generar reporte)
-- Este paso se ejecuta después de un tiempo de monitoreo
SELECT
    event_data.value('(event/@name)[1]', 'varchar(50)') AS event_name,
    event_data.value('(event/data[@name="duration"]/value)[1]', 'bigint') /
1000 AS duration_ms,
    event_data.value('(event/data[@name="cpu_time"]/value)[1]', 'bigint') /
1000 AS cpu_ms,
    event_data.value('(event/data[@name="logical_reads"]/value)[1]', 'bigint')
AS logical_reads,
    event_data.value('(event/data[@name="writes"]/value)[1]', 'bigint') AS
writes,
    event_data.value('(event/action[@name="sql_text"]/value)[1]',
'nvarchar(max)') AS sql_text,
    event_data.value('(event/action[@name="username"]/value)[1]',
'varchar(100)') AS username,
    event_data.value('(event/action[@name="session_id"]/value)[1]', 'int') AS
session_id
FROM (
    SELECT CAST(event_data AS XML) AS event_data
    FROM
sys.fn_xe_file_target_read_file('D:\BaseDatos2026\MonitorConsultasLentas_Grados
Titulos*.xel', NULL, NULL, NULL)
) AS ed
ORDER BY duration_ms DESC;
GO

-- 5. Generar reporte resumen (exportar a tabla o archivo)
-- Creamos una tabla para almacenar el historial
USE GradosTitulos;
GO

IF OBJECT_ID('Auditoria.ConsultasLentas', 'U') IS NOT NULL
    DROP TABLE Auditoria.ConsultasLentas;
GO

CREATE SCHEMA Auditoria; -- Si no existe
GO

CREATE TABLE Auditoria.ConsultasLentas (

```

```

        Id                INT IDENTITY(1,1) PRIMARY KEY,
        FechaCaptura     DATETIME2 DEFAULT SYSDATETIME(),
        Evento            VARCHAR(50),
        DuracionMs        BIGINT,
        CpuMs             BIGINT,
        LecturasLogicas   BIGINT,
        Escrituras        BIGINT,
        TextoSql          NVARCHAR(MAX),
        Usuario           VARCHAR(100),
        SesionId          INT
    );
GO

-- Insertar datos capturados en la tabla
INSERT INTO Auditoria.ConsultasLentas (
    Evento, DuracionMs, CpuMs, LecturasLogicas, Escrituras, TextoSql, Usuario,
    SesionId
)
SELECT
    event_data.value('(event/@name)[1]', 'varchar(50)') AS event_name,
    event_data.value('(event/data[@name="duration"]/value)[1]', 'bigint') /
1000 AS duration_ms,
    event_data.value('(event/data[@name="cpu_time"]/value)[1]', 'bigint') /
1000 AS cpu_ms,
    event_data.value('(event/data[@name="logical_reads"]/value)[1]', 'bigint')
AS logical_reads,
    event_data.value('(event/data[@name="writes"]/value)[1]', 'bigint') AS
writes,
    event_data.value('(event/action[@name="sql_text"]/value)[1]',
'nvarchar(max)') AS sql_text,
    event_data.value('(event/action[@name="username"]/value)[1]',
'varchar(100)') AS username,
    event_data.value('(event/action[@name="session_id"]/value)[1]', 'int') AS
session_id
FROM (
    SELECT CAST(event_data AS XML) AS event_data
    FROM
sys.fn_xe_file_target_read_file('D:\BaseDatos2026\MonitorConsultasLentas_Grados
Titulos*.xel', NULL, NULL, NULL)
) AS ed
WHERE NOT EXISTS (
    SELECT 1 FROM Auditoria.ConsultasLentas AS cl
    WHERE cl.TextoSql =
ed.event_data.value('(event/action[@name="sql_text"]/value)[1]',
'nvarchar(max)')
    AND cl.DuracionMs =
event_data.value('(event/data[@name="duration"]/value)[1]', 'bigint') / 1000
);
GO

-- 6. Reporte final: top 10 consultas más lentas
SELECT TOP 10
    Evento,
    DuracionMs,
    CpuMs,
    LecturasLogicas,
    Escrituras,
    LEFT(TextoSql, 200) AS TextoResumido,
    Usuario,
    FechaCaptura
FROM Auditoria.ConsultasLentas
ORDER BY DuracionMs DESC;

```

GO

```
-- 7. Detener la sesión cuando termine el monitoreo
-- ALTER EVENT SESSION [MonitorConsultasLentas_GradosTitulos] ON SERVER STATE =
STOP;
-- GO
```

Proyecto 2: Implementación de Backups Diferenciales para Trámites

Académicos Enunciado: Durante los procesos de matrícula para obtención del título profesional, varios usuarios reportan tiempos elevados en la generación de resoluciones.

Codigo:

```
-- =====
-- PROYECTO 2: Monitoreo avanzado (lentas, deadlocks, bloqueos,
--               errores > 16, alto CPU)
-- =====

USE master;
GO

IF EXISTS (SELECT * FROM sys.server_event_sessions WHERE name =
'MonitorAvanzado_GradosTitulos')
    DROP EVENT SESSION [MonitorAvanzado_GradosTitulos] ON SERVER;
GO

CREATE EVENT SESSION [MonitorAvanzado_GradosTitulos] ON SERVER
ADD EVENT sqlserver.sql_batch_completed
(
    ACTION (sqlserver.sql_text, sqlserver.database_id, sqlserver.username,
sqlserver.session_id)
    WHERE (
        [database_id] = DB_ID('GradosTitulos')
        AND (
            [duration] > 1000000 -- > 1s
            OR [cpu_time] > 500000 -- > 500 ms de CPU
        )
    )
)
ADD EVENT sqlserver.rpc_completed
(
    ACTION (sqlserver.sql_text, sqlserver.database_id, sqlserver.username,
sqlserver.session_id)
    WHERE (
        [database_id] = DB_ID('GradosTitulos')
        AND (
            [duration] > 1000000
            OR [cpu_time] > 500000
        )
    )
)
ADD EVENT sqlserver.lock_acquired
(
    ACTION (sqlserver.sql_text, sqlserver.database_id, sqlserver.username,
sqlserver.session_id)
    WHERE (
        [database_id] = DB_ID('GradosTitulos')
        AND [mode] IN ('X', 'U', 'IX')
        AND [resource_description] LIKE '%PAGE%' -- bloqueos a nivel página
    )
)
ADD EVENT sqlserver.lock_released
```

```

(
    ACTION (sqlserver.sql_text, sqlserver.database_id, sqlserver.username,
sqlserver.session_id)
    WHERE (
        [database_id] = DB_ID('GradosTitulos')
    )
)
)
ADD EVENT sqlserver.deadlock_graph
(
    ACTION (sqlserver.sql_text, sqlserver.database_id, sqlserver.username,
sqlserver.session_id)
    WHERE [database_id] = DB_ID('GradosTitulos')
)
)
ADD EVENT sqlserver.error_reported
(
    ACTION (sqlserver.sql_text, sqlserver.database_id, sqlserver.username,
sqlserver.session_id)
    WHERE (
        [database_id] = DB_ID('GradosTitulos')
        AND [severity] > 16
    )
)
)
ADD TARGET package0.event_file
(
    SET filename = N'D:\BaseDatos2026\MonitorAvanzado_GradosTitulos.xel',
        max_file_size = 100, -- MB
        max_rollover_files = 10
)
)
WITH (
    MAX_MEMORY = 8192 KB,
    EVENT_RETENTION_MODE = ALLOW_SINGLE_EVENT_LOSS,
    MAX_DISPATCH_LATENCY = 30 SECONDS,
    MAX_EVENT_SIZE = 0 KB,
    MEMORY_PARTITION_MODE = NONE,
    TRACK_CAUSALITY = OFF,
    STARTUP_STATE = OFF
);
GO

-- Iniciar sesión
ALTER EVENT SESSION [MonitorAvanzado_GradosTitulos] ON SERVER STATE = START;
GO

-- Verificar estado
SELECT name, state_desc FROM sys.server_event_sessions WHERE name =
'MonitorAvanzado_GradosTitulos';
GO

-- Consultar eventos capturados (ejemplo)
SELECT
    event_data.value('(event/@name)[1]', 'varchar(50)') AS event_name,
    event_data.value('(event/data[@name="duration"]/value)[1]', 'bigint') /
1000 AS duration_ms,
    event_data.value('(event/data[@name="cpu_time"]/value)[1]', 'bigint') /
1000 AS cpu_ms,
    event_data.value('(event/data[@name="logical_reads"]/value)[1]', 'bigint')
AS reads,
    event_data.value('(event/action[@name="sql_text"]/value)[1]',
'nvarchar(max)') AS sql_text,
    event_data.value('(event/action[@name="username"]/value)[1]',
'varchar(100)') AS username
FROM (

```

```

        SELECT CAST(event_data AS XML) AS event_data
        FROM
sys.fn_xe_file_target_read_file('D:\BaseDatos2026\MonitorAvanzado_GradosTitulos
*.xel', NULL, NULL, NULL)
) AS ed
ORDER BY event_data.value('(event/data[@name="duration"]/value)[1]', 'bigint')
DESC;
GO

```

Proyecto 3: Estrategia Integral Full + Differential + Log Backup Enunciado: La universidad desea implementar un sistema permanente de diagnóstico del rendimiento para la base de datos GradosTitulos.

Codigo:

```

-- =====
-- PROYECTO 3: Monitoreo permanente con tabla histórica
-- =====

-- 1. Crear esquema y tabla de auditoría (si no existen)
USE GradosTitulos;
GO

IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'Auditoria')
    EXEC ('CREATE SCHEMA Auditoria');
GO

IF OBJECT_ID('Auditoria.ConsultasCostosas', 'U') IS NULL
BEGIN
    CREATE TABLE Auditoria.ConsultasCostosas (
        Id                INT IDENTITY(1,1) PRIMARY KEY,
        FechaCaptura      DATETIME2 DEFAULT SYSDATETIME(),
        Evento             VARCHAR(50),
        DuracionMs         BIGINT,
        CpuMs              BIGINT,
        LecturasLogicas    BIGINT,
        Escrituras          BIGINT,
        TextoSql           NVARCHAR(MAX),
        Usuario            VARCHAR(100),
        SesionId           INT,
        Procesado          BIT DEFAULT 0 -- para saber si ya se analizó
    );
END
GO

-- 2. Crear la sesión de Extended Events (enfocada en consultas costosas)
USE master;
GO

IF EXISTS (SELECT * FROM sys.server_event_sessions WHERE name =
'MonitorPermanente_GradosTitulos')
    DROP EVENT SESSION [MonitorPermanente_GradosTitulos] ON SERVER;
GO

CREATE EVENT SESSION [MonitorPermanente_GradosTitulos] ON SERVER
ADD EVENT sqlserver.sql_batch_completed
(
    ACTION (sqlserver.sql_text, sqlserver.database_id, sqlserver.username,
sqlserver.session_id)
    WHERE (
        [database_id] = DB_ID('GradosTitulos')
        AND (
            [duration] > 500000 -- 500 ms, umbral más bajo para capturar más

```

```

        OR [cpu_time] > 300000
        OR [logical_reads] > 100000
    )
)
)
)
ADD EVENT sqlserver.rpc_completed
(
    ACTION (sqlserver.sql_text, sqlserver.database_id, sqlserver.username,
sqlserver.session_id)
    WHERE (
        [database_id] = DB_ID('GradosTitulos')
        AND (
            [duration] > 500000
            OR [cpu_time] > 300000
            OR [logical_reads] > 100000
        )
    )
)
)
ADD TARGET package0.event_file
(
    SET filename = N'D:\BaseDatos2026\MonitorPermanente_GradosTitulos.xel',
        max_file_size = 50,
        max_rollover_files = 20
)
WITH (
    MAX_MEMORY = 4096 KB,
    EVENT_RETENTION_MODE = ALLOW_SINGLE_EVENT_LOSS,
    MAX_DISPATCH_LATENCY = 30 SECONDS,
    MAX_EVENT_SIZE = 0 KB,
    MEMORY_PARTITION_MODE = NONE,
    TRACK_CAUSALITY = OFF,
    STARTUP_STATE = OFF
);
GO

-- 3. Iniciar la sesión (se puede configurar para que inicie automáticamente al
arrancar SQL Server)
ALTER EVENT SESSION [MonitorPermanente_GradosTitulos] ON SERVER STATE = START;
GO

-- 4. Procedimiento almacenado para importar datos del archivo de eventos a la
tabla histórica
USE GradosTitulos;
GO

CREATE OR ALTER PROCEDURE Auditoria.SP_ImportarConsultasCostosas
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        -- Insertar eventos nuevos que no estén ya en la tabla
        INSERT INTO Auditoria.ConsultasCostosas (
            Evento, DuracionMs, CpuMs, LecturasLogicas, Escrituras,
            TextoSql, Usuario, SesionId, Procesado
        )
        SELECT
            event_data.value('(event/@name)[1]', 'varchar(50)') AS event_name,
            event_data.value('(event/data[@name="duration"]/value)[1]',
'bigint') / 1000 AS duration_ms,
            event_data.value('(event/data[@name="cpu_time"]/value)[1]',
'bigint') / 1000 AS cpu_ms,

```



```

        event_data.value('(event/data[@name="logical_reads"]/value)[1]',
'bigint') AS logical_reads,
        event_data.value('(event/data[@name="writes"]/value)[1]', 'bigint')
AS writes,
        event_data.value('(event/action[@name="sql_text"]/value)[1]',
'nvarchar(max)') AS sql_text,
        event_data.value('(event/action[@name="username"]/value)[1]',
'varchar(100)') AS username,
        event_data.value('(event/action[@name="session_id"]/value)[1]',
'int') AS session_id,
        0 -- Procesado = 0 (pendiente)
FROM (
        SELECT CAST(event_data AS XML) AS event_data
        FROM
sys.fn_xe_file_target_read_file('D:\BaseDatos2026\MonitorPermanente_GradosTitul
os*.xel', NULL, NULL, NULL)
) AS ed
WHERE NOT EXISTS (
        SELECT 1 FROM Auditoria.ConsultasCostosas AS cc
        WHERE cc.TextoSql =
ed.event_data.value('(event/action[@name="sql_text"]/value)[1]',
'nvarchar(max)')
        AND cc.DuracionMs =
event_data.value('(event/data[@name="duration"]/value)[1]', 'bigint') / 1000
        AND cc.FechaCaptura > DATEADD(hour, -1, SYSDATETIME()) -- evitar
duplicados recientes
);

```

```

        PRINT 'Se importaron ' + CAST(@@ROWCOUNT AS VARCHAR(10)) + ' nuevos
registros.';

```

```

        END TRY
        BEGIN CATCH
                PRINT 'Error al importar: ' + ERROR_MESSAGE();
        END CATCH
END
GO

```

```

-- 5. Crear un trabajo del Agente SQL para ejecutar la importación cada 15
minutos (opcional)
-- (Esto requiere que el Agente SQL esté habilitado)
-- EXEC msdb.dbo.sp_add_job ... etc.

```

```

-- 6. Generar reporte de consultas con mayor duración, CPU y lecturas lógicas
CREATE OR ALTER PROCEDURE Auditoria.SP_ReporteConsultasCostosas

```

```

    @TopN INT = 10

```

```

AS

```

```

BEGIN

```

```

    SET NOCOUNT ON;

```

```

    -- Top N por duración

```

```

    SELECT TOP (@TopN)
        'Duracion' AS Criterio,
        DuracionMs AS Valor,
        TextoSql,
        Usuario,
        FechaCaptura
    FROM Auditoria.ConsultasCostosas
    WHERE Procesado = 0 -- solo las nuevas
    ORDER BY DuracionMs DESC;

```

```

    -- Top N por CPU

```

```

    SELECT TOP (@TopN)

```

```

        'CPU' AS Criterio,
        CpuMs AS Valor,
        TextoSql,
        Usuario,
        FechaCaptura
FROM Auditoria.ConsultasCostosas
WHERE Procesado = 0
ORDER BY CpuMs DESC;

-- Top N por lecturas lógicas
SELECT TOP (@TopN)
    'LecturasLogicas' AS Criterio,
    LecturasLogicas AS Valor,
    TextoSql,
    Usuario,
    FechaCaptura
FROM Auditoria.ConsultasCostosas
WHERE Procesado = 0
ORDER BY LecturasLogicas DESC;

-- Marcar como procesados los registros mostrados
UPDATE Auditoria.ConsultasCostosas
SET Procesado = 1
WHERE Id IN (
    SELECT TOP (@TopN * 3) Id
    FROM Auditoria.ConsultasCostosas
    WHERE Procesado = 0
    ORDER BY DuracionMs DESC
);
END
GO

-- 7. Ejecutar el reporte manualmente
EXEC Auditoria.SP_ReporteConsultasCostosas @TopN = 5;
GO

-- 8. Detener la sesión cuando sea necesario
-- ALTER EVENT SESSION [MonitorPermanente_GradosTitulos] ON SERVER STATE = STOP;
-- GO

```

TEMA 2. Estadísticas e índices

Proyecto 1: Optimización de índices para consultas de estudiantes Enunciado

Las consultas realizadas sobre las tablas **Academico.Persona**, **Academico.Matricula** y **Tramite.Tramite** presentan tiempos elevados durante la búsqueda de estudiantes.

Codigo:

```

-- =====
-- PROYECTO 1: Optimización de índices
-- Tablas objetivo: Persona, Matricula, Tramite
-- =====

USE GradosTitulos;
GO

-- 1. ANALIZAR ÍNDICES FALTANTES SUGERIDOS POR EL MOTOR (DMV)
-- Esta consulta muestra recomendaciones del optimizador basadas en el
historial de ejecución
SELECT

```

```

        migs.avg_total_user_cost * migs.avg_user_impact * (migs.user_seeks +
migs.user_scans) AS Impacto,
        mid.statement AS Tabla,
        mid.equality_columns AS Columnas_Igualdad,
        mid.inequality_columns AS Columnas_Desigualdad,
        mid.included_columns AS Columnas_Incluidas,
        'CREATE INDEX [IX_Missing_' +
        REPLACE(REPLACE(REPLACE(mid.equality_columns, '[', ''), ']', ''), ', ',
'_' ) +
        CASE WHEN mid.inequality_columns IS NOT NULL THEN '_IN_' ELSE '' END +
        REPLACE(REPLACE(REPLACE(mid.inequality_columns, '[', ''), ']', ''), ', ',
'_' ) +
        ']' ON ' + mid.statement + ' (' +
        ISNULL(mid.equality_columns, '') +
        CASE WHEN mid.equality_columns IS NOT NULL AND mid.inequality_columns
IS NOT NULL THEN ',' ELSE '' END +
        ISNULL(mid.inequality_columns, '') +
        ') ' +
        CASE WHEN mid.included_columns IS NOT NULL THEN 'INCLUDE (' +
mid.included_columns + ')' ELSE '' END
        AS Comando_Creacion
FROM sys.dm_db_missing_index_groups mig
INNER JOIN sys.dm_db_missing_index_group_stats migs
        ON migs.group_handle = mig.index_group_handle
INNER JOIN sys.dm_db_missing_index_details mid
        ON mig.index_handle = mid.index_handle
WHERE mid.database_id = DB_ID('GradosTitulos')
        AND mid.statement LIKE '%Academico.Persona%'
        OR mid.statement LIKE '%Academico.Matricula%'
        OR mid.statement LIKE '%Tramite.Tramite%'
ORDER BY Impacto DESC;
GO

```

-- 2. CREAR ÍNDICES RECOMENDADOS MANUALMENTE (Basados en la estructura y lógica de negocio)

-- 2.1 Tabla Persona

-- Ya existe IX_Persona_DNI. Agregamos índices para búsquedas por nombre y apellido,
-- así como un índice cubriente para consultas que devuelvan correo/telefono.

```

IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name =
'IX_Persona_ApellidoPaterno_Nombres' AND object_id =
OBJECT_ID('Academico.Persona'))

```

BEGIN

```

        CREATE NONCLUSTERED INDEX IX_Persona_ApellidoPaterno_Nombres
        ON Academico.Persona (ApellidoPaterno, Nombres)
        INCLUDE (DNI, Correo, Telefono, TipoPersona);
        PRINT 'Índice IX_Persona_ApellidoPaterno_Nombres creado.';

```

END

GO

```

IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = 'IX_Persona_Tipo_DNI' AND
object_id = OBJECT_ID('Academico.Persona'))

```

BEGIN

```

        CREATE NONCLUSTERED INDEX IX_Persona_Tipo_DNI
        ON Academico.Persona (TipoPersona, DNI);
        PRINT 'Índice IX_Persona_Tipo_DNI creado.';

```

END

GO

-- 2.2 Tabla Matricula

```

-- Ya existe IX_Matricula_Estudiante. Agregamos un índice compuesto para
filtrar por estudiante y estado de egreso.
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name =
'IX_Matricula_Estudiante_Egreso' AND object_id =
OBJECT_ID('Academico.Matricula'))
BEGIN
    CREATE NONCLUSTERED INDEX IX_Matricula_Estudiante_Egreso
    ON Academico.Matricula (IdEstudiante, FechaEgreso)
    INCLUDE (IdPrograma, CreditosAprobados, TieneDeuda);
    PRINT 'Índice IX_Matricula_Estudiante_Egreso creado.';
END
GO

IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name =
'IX_Matricula_Programa_TieneDeuda' AND object_id =
OBJECT_ID('Academico.Matricula'))
BEGIN
    CREATE NONCLUSTERED INDEX IX_Matricula_Programa_TieneDeuda
    ON Academico.Matricula (IdPrograma, TieneDeuda)
    INCLUDE (IdEstudiante, FechaEgreso);
    PRINT 'Índice IX_Matricula_Programa_TieneDeuda creado.';
END
GO

-- 2.3 Tabla Tramite
-- Ya existe IX_Tramite_Matricula y IX_Tramite_Estado. Agregamos índice
cubriente para consultas de seguimiento.
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name =
'IX_Tramite_Matricula_Estado_Fecha' AND object_id =
OBJECT_ID('Tramite.Tramite'))
BEGIN
    CREATE NONCLUSTERED INDEX IX_Tramite_Matricula_Estado_Fecha
    ON Tramite.Tramite (IdMatricula, IdEstadoTramite, FechaInicio)
    INCLUDE (IdTipoTramite, Observaciones);
    PRINT 'Índice IX_Tramite_Matricula_Estado_Fecha creado.';
END
GO

IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name =
'IX_Tramite_Estado_Tipo_Fecha' AND object_id = OBJECT_ID('Tramite.Tramite'))
BEGIN
    CREATE NONCLUSTERED INDEX IX_Tramite_Estado_Tipo_Fecha
    ON Tramite.Tramite (IdEstadoTramite, IdTipoTramite, FechaUltMod)
    INCLUDE (Observaciones);
    PRINT 'Índice IX_Tramite_Estado_Tipo_Fecha creado.';
END
GO

-- 3. ACTUALIZAR ESTADÍSTICAS DE LAS TABLAS CRÍTICAS
-- Las estadísticas actualizadas ayudan al optimizador a elegir mejores planes
de ejecución.
PRINT 'Actualizando estadísticas de Academico.Persona...';
UPDATE STATISTICS Academico.Persona WITH FULLSCAN;
GO

PRINT 'Actualizando estadísticas de Academico.Matricula...';
UPDATE STATISTICS Academico.Matricula WITH FULLSCAN;
GO

PRINT 'Actualizando estadísticas de Tramite.Tramite...';
UPDATE STATISTICS Tramite.Tramite WITH FULLSCAN;
GO

```

```

-- 4. VERIFICAR ESTADÍSTICAS ACTUALIZADAS (fecha de última actualización)
SELECT
    OBJECT_NAME(s.object_id) AS Tabla,
    s.name AS Estadistica,
    STATS_DATE(s.object_id, s.stats_id) AS Fecha_Ultima_Actualizacion,
    s.auto_created AS Creada_Automaticamente
FROM sys.stats s
WHERE OBJECT_NAME(s.object_id) IN ('Persona', 'Matricula', 'Tramite')
ORDER BY Tabla, Fecha_Ultima_Actualizacion DESC;
GO

-- 5. REPORTE FINAL: Índices existentes en las tablas objetivo
PRINT '=====';
PRINT 'REPORTE DE ÍNDICES EN TABLAS OBJETIVO';
PRINT '=====';
SELECT
    t.name AS Tabla,
    i.name AS Indice,
    i.type_desc AS Tipo,
    COUNT(DISTINCT ic.column_id) AS Columnas_Clave,
    STRING_AGG(c.name, ', ') WITHIN GROUP (ORDER BY ic.key_ordinal) AS
Columnas_Clave_Lista
FROM sys.tables t
JOIN sys.indexes i ON t.object_id = i.object_id
JOIN sys.index_columns ic ON i.object_id = ic.object_id AND i.index_id =
ic.index_id
JOIN sys.columns c ON ic.object_id = c.object_id AND ic.column_id = c.column_id
WHERE t.name IN ('Persona', 'Matricula', 'Tramite')
    AND i.type > 0 -- Excluir HEAP
    AND i.is_primary_key = 0
    AND ic.key_ordinal > 0
GROUP BY t.name, i.name, i.type_desc
ORDER BY t.name, i.name;

GO

```

Proyecto 2: Detección y corrección de fragmentación Enunciado Después de varios meses de operaciones se observa disminución del rendimiento debido al crecimiento de las tablas.

Codigo:

```

-- =====
-- PROYECTO 2: Detección y corrección de fragmentación
-- Aplica a TODAS las tablas de los esquemas: Academico, Tramite,
-- Evaluacion, Documento
-- =====

USE GradosTitulos;
GO

-- 1. ANALIZAR FRAGMENTACIÓN ACTUAL
-- Muestra el porcentaje de fragmentación, tamaño y tipo de índice.
SELECT
    DB_NAME() AS BaseDatos,
    OBJECT_SCHEMA_NAME(ips.object_id) AS Esquema,
    OBJECT_NAME(ips.object_id) AS Tabla,
    i.name AS Indice,
    i.type_desc AS TipoIndice,

```

```

ips.avg_fragmentation_in_percent AS Fragmentacion_Porcentaje,
ips.page_count AS Numero_Paginas,
ips.record_count AS Numero_Registros,
ips.fragment_count AS Numero_Fragmentos,
CASE
    WHEN ips.avg_fragmentation_in_percent > 30 THEN 'REBUILD (Alta)'
    WHEN ips.avg_fragmentation_in_percent > 5 THEN 'REORGANIZE (Media)'
    ELSE 'OK (Baja)'
END AS Accion_Recomendada
FROM sys.dm_db_index_physical_stats(
    DB_ID('GradosTitulos'), -- database_id
    NULL, -- object_id (todos)
    NULL, -- index_id (todos)
    NULL, -- partition_id
    'LIMITED' -- modo rápido
) ips
INNER JOIN sys.indexes i ON ips.object_id = i.object_id AND ips.index_id =
i.index_id
WHERE ips.avg_fragmentation_in_percent > 0
    AND i.type > 0 -- No HEAPs
    AND OBJECT_SCHEMA_NAME(ips.object_id) IN ('Academico', 'Tramite',
'Evaluacion', 'Documento')
    AND ips.page_count > 100 -- Solo tablas con más de 100 páginas para no
saturar
ORDER BY ips.avg_fragmentation_in_percent DESC, ips.page_count DESC;
GO

-- 2. PROCEDIMIENTO AUTOMATIZADO PARA REORGANIZAR / REBUILD
-- Ejecuta REORGANIZE si fragmentación entre 5% y 30%, REBUILD si > 30%.
-- Nota: En entornos productivos, REBUILD suele programarse en horas de bajo
uso.
CREATE OR ALTER PROCEDURE Administracion.SP_MantenerIndices
    @RebuildThreshold FLOAT = 30.0,
    @ReorganizeThreshold FLOAT = 5.0,
    @MaxPages INT = 100 -- Ignorar índices muy pequeños
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE
        @ObjectID INT,
        @IndexID INT,
        @PartitionNum BIGINT,
        @Fragmentation FLOAT,
        @PageCount INT,
        @Command NVARCHAR(1000),
        @SchemaName NVARCHAR(100),
        @TableName NVARCHAR(200),
        @IndexName NVARCHAR(200);

    -- Crear tabla temporal para almacenar índices con fragmentación
    IF OBJECT_ID('tempdb..#FragIndexes') IS NOT NULL DROP TABLE #FragIndexes;

    SELECT
        ips.object_id,
        ips.index_id,
        ips.partition_number,
        ips.avg_fragmentation_in_percent AS Frag,
        ips.page_count,
        OBJECT_SCHEMA_NAME(ips.object_id) AS Esquema,
        OBJECT_NAME(ips.object_id) AS Tabla,
        i.name AS Indice
    INTO #FragIndexes

```

```

FROM sys.dm_db_index_physical_stats(DB_ID('GradosTitulos'), NULL, NULL,
NULL, 'LIMITED') ips
JOIN sys.indexes i ON ips.object_id = i.object_id AND ips.index_id =
i.index_id
WHERE i.type > 0 -- Excluir HEAP
AND OBJECT_SCHEMA_NAME(ips.object_id) IN ('Academico', 'Tramite',
'Evaluacion', 'Documento')
AND ips.avg_fragmentation_in_percent > @ReorganizeThreshold
AND ips.page_count > @MaxPages;

-- Cursor para procesar cada índice
DECLARE idx_cursor CURSOR FOR
SELECT object_id, index_id, partition_number, Frag, page_count, Esquema,
Tabla, Indice
FROM #FragIndexes
ORDER BY Frag DESC;

OPEN idx_cursor;
FETCH NEXT FROM idx_cursor INTO @ObjectID, @IndexID, @PartitionNum,
@Fragmentation, @PageCount, @SchemaName, @TableName, @IndexName;

WHILE @@FETCH_STATUS = 0
BEGIN
IF @Fragmentation > @RebuildThreshold
BEGIN
SET @Command = 'ALTER INDEX [' + @IndexName + '] ON [' +
@SchemaName + '].[' + @TableName + '] REBUILD PARTITION = ' +
CAST(@PartitionNum AS NVARCHAR(10)) + ' WITH (ONLINE = ON, SORT_IN_TEMPDB =
ON);';
PRINT 'REBUILD: ' + @Command;
END
ELSE IF @Fragmentation > @ReorganizeThreshold
BEGIN
SET @Command = 'ALTER INDEX [' + @IndexName + '] ON [' +
@SchemaName + '].[' + @TableName + '] REORGANIZE PARTITION = ' +
CAST(@PartitionNum AS NVARCHAR(10)) + ';';
PRINT 'REORGANIZE: ' + @Command;
END

-- Ejecutar el comando
BEGIN TRY
EXEC sp_executesql @Command;
PRINT '--> Ejecutado correctamente.';
END TRY
BEGIN CATCH
PRINT '!!! ERROR: ' + ERROR_MESSAGE();
END CATCH

FETCH NEXT FROM idx_cursor INTO @ObjectID, @IndexID, @PartitionNum,
@Fragmentation, @PageCount, @SchemaName, @TableName, @IndexName;
END

CLOSE idx_cursor;
DEALLOCATE idx_cursor;
DROP TABLE #FragIndexes;
END
GO

-- Crear esquema Administracion si no existe
IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'Administracion')
EXEC ('CREATE SCHEMA Administracion');
GO

```

```

-- 3. EJECUTAR EL MANTENIMIENTO DE ÍNDICES (Puede tomar varios minutos)
-- Se recomienda ejecutar en horas de bajo tráfico.
EXEC Administracion.SP_MantenerIndices
    @RebuildThreshold = 30.0,
    @ReorganizeThreshold = 5.0,
    @MaxPages = 100;
GO

-- 4. VERIFICAR FRAGMENTACIÓN DESPUÉS DEL MANTENIMIENTO
SELECT
    OBJECT_SCHEMA_NAME(ips.object_id) AS Esquema,
    OBJECT_NAME(ips.object_id) AS Tabla,
    i.name AS Indice,
    ips.avg_fragmentation_in_percent AS Fragmentacion_Porcentaje,
    ips.page_count AS Numero_Paginas,
    CASE
        WHEN ips.avg_fragmentation_in_percent > 30 THEN 'REQUIERE REBUILD'
        WHEN ips.avg_fragmentation_in_percent > 5 THEN 'REQUIERE REORGANIZE'
        ELSE 'OK'
    END AS Estado
FROM sys.dm_db_index_physical_stats(DB_ID('GradosTitulos'), NULL, NULL, NULL,
'LIMITED') ips
INNER JOIN sys.indexes i ON ips.object_id = i.object_id AND ips.index_id =
i.index_id
WHERE i.type > 0
    AND OBJECT_SCHEMA_NAME(ips.object_id) IN ('Academico', 'Tramite',
'Evaluacion', 'Documento')
    AND ips.avg_fragmentation_in_percent > 1
    AND ips.page_count > 50
ORDER BY ips.avg_fragmentation_in_percent DESC;
GO

-- 5. CREAR UN TRABAJO PROGRAMADO (OPCIONAL) EN EL AGENTE SQL
-- Ejecutar este bloque si se desea automatizar el mantenimiento semanal.
/*
USE msdb;
GO
EXEC dbo.sp_add_job
    @job_name = N'Mantenimiento_Indices_GradosTitulos';
EXEC sp_add_jobstep
    @job_name = N'Mantenimiento_Indices_GradosTitulos',
    @step_name = N'Ejecutar_REBUILD_REORGANIZE',
    @command = N'EXEC GradosTitulos.Administracion.SP_MantenerIndices';
EXEC sp_add_schedule
    @schedule_name = N'Weekly_Sunday_2AM',
    @freq_type = 8,
    @freq_interval = 1, -- Domingos
    @active_start_time = 020000;
EXEC sp_attach_schedule
    @job_name = N'Mantenimiento_Indices_GradosTitulos',
    @schedule_name = N'Weekly_Sunday_2AM';
EXEC dbo.sp_add_jobserver
    @job_name = N'Mantenimiento_Indices_GradosTitulos';
GO
*/

```


TEMA 3. Administración de transacciones y bloqueos

Proyecto 1: Control de transacciones en la aprobación de títulos Enunciado

Durante la aprobación de un título profesional intervienen varias tablas como

Tramite, Resolucion, Diploma y EvaluacionTrabajo.

Codigo:

--

=====

-- PROYECTO 1: Control de transacciones en aprobación de títulos

-- Involucra: Tramite.Tramite, Documento.Resolucion,

-- Documento.Diploma, Evaluacion.EvaluacionTrabajo

--

=====

USE GradosTitulos;

GO

-- 1. CREAR ESQUEMA ADMINISTRACION SI NO EXISTE

IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'Administracion')

EXEC ('CREATE SCHEMA Administracion');

GO

-- 2. PROCEDIMIENTO PARA APROBACIÓN DE TÍTULO (TRANSACCIONAL)

CREATE OR ALTER PROCEDURE Administracion.SP_AprobarTituloProfesional

@IdTramite INT,

@NroResolucion VARCHAR(50),

@FechaEmision DATE,

@OrganoEmisor VARCHAR(25),

```

        @NroDiploma VARCHAR(40),

        @FirmanteRector VARCHAR(150),

        @FirmanteDecano VARCHAR(150),

        @FirmanteSecretario VARCHAR(150),

        @Observaciones VARCHAR(500) = NULL,

        @IdResolucion INT OUTPUT,

        @IdDiploma INT OUTPUT,

        @ErrorMsg NVARCHAR(4000) = NULL OUTPUT
AS
BEGIN

    SET NOCOUNT ON;

    SET XACT_ABORT OFF; -- Control manual del rollback


    DECLARE @IdMatricula INT, @IdTipoTramite TINYINT, @IdEstadoActual
    TINYINT;

    DECLARE @TranName VARCHAR(32) = 'Tran_AprobacionTitulo';


    BEGIN TRY

        -- 1. Validar que el trámite existe y está en estado APROBADO (5)

        SELECT

            @IdMatricula = IdMatricula,

            @IdTipoTramite = IdTipoTramite,

            @IdEstadoActual = IdEstadoTramite

        FROM Tramite.Tramite

        WHERE IdTramite = @IdTramite;

```

```

IF @IdMatricula IS NULL

    THROW 50001, 'El trámite especificado no existe.', 1;


IF @IdEstadoActual != 5 -- 5 = APROBADO

    THROW 50002, 'El trámite no está en estado APROBADO (5). Estado actual: ' + CAST(@IdEstadoActual AS VARCHAR), 1;


-- 2. Verificar que el tipo de trámite sea TITULO (2, 3 o 4)

IF @IdTipoTramite NOT IN (2, 3, 4)

    THROW 50003, 'El trámite no corresponde a un título profesional (Tipos 2, 3 o 4).', 1;


-- 3. Validar que exista evaluación aprobada (nota >= 13)

IF NOT EXISTS (

    SELECT 1

    FROM Evaluacion.Sustentacion s

    INNER JOIN Tramite.TrabajoInvestigacion ti ON ti.IdTramite = @IdTramite

    WHERE s.IdTrabajo = ti.IdTrabajo

    AND s.Condicion = 'APROBADO'

)

    THROW 50004, 'El trámite no tiene una sustentación aprobada.', 1;


-- 4. Iniciar transacción

BEGIN TRANSACTION @TranName;


-- 5. Crear la resolución

```

```
INSERT INTO Documento.Resolucion (
    IdTramite, TipoResolucion, NroResolucion, FechaEmision, OrganoEmisor,
    Descripcion
)
VALUES (
    @IdTramite,
    'APROBACION_TITULO',
    @NroResolucion,
    @FechaEmision,
    @OrganoEmisor,
    @Observaciones
);
```

```
SET @IdResolucion = SCOPE_IDENTITY();
```

```
-- 6. Crear el diploma
```

```
INSERT INTO Documento.Diploma (
    IdTramite,
    TipoDiploma,
    NroDiploma,
    FechaExpedicion,
    FechaRegistroSunedu,
    FirmanteRector,
    FirmanteDecano,
    FirmanteSecretario,
    EsDuplicado,
```

```

    FechaDuplicado,

    MotivosDuplicado

)

VALUES (

    @IdTramite,

    'TITULO_PROFESIONAL',

    @NroDiploma,

    @FechaEmision, -- Misma fecha que resolución

    NULL,          -- Pendiente de registro SUNEDU

    @FirmanteRector,

    @FirmanteDecano,

    @FirmanteSecretario,

    0,             -- No es duplicado

    NULL,

    NULL

);

```

```

SET @IdDiploma = SCOPE_IDENTITY();

```

```

-- 7. Actualizar el estado del trámite a terminal (por ejemplo, 7 = ARCHIVADO)

```

```

-- NOTA: Podría dejarse en APROBADO (5) y crear una vista para ver los
finalizados

```

```

UPDATE Tramite.Tramite

```

```

SET

```

```

    IdEstadoTramite = 7, -- ARCHIVADO

```

```

    FechaUltMod = SYSDATETIME(),

```

```

Observaciones = COALESCE(Observaciones, '') +

CASE WHEN Observaciones IS NOT NULL THEN '|' ELSE '' END +

'Título aprobado con Resolución ' + @NroResolucion + ' y Diploma ' +
@NroDiploma

WHERE IdTramite = @IdTramite;

-- 8. Confirmar transacción

COMMIT TRANSACTION @TranName;

-- 9. Devolver mensaje de éxito

SET @ErrorMsg = 'Título aprobado exitosamente. Resolución ID: ' +
CAST(@IdResolucion AS VARCHAR) +

', Diploma ID: ' + CAST(@IdDiploma AS VARCHAR);

PRINT @ErrorMsg;

END TRY

BEGIN CATCH

-- 10. Rollback en caso de error

IF @@TRANCOUNT > 0

ROLLBACK TRANSACTION @TranName;

-- 11. Capturar error detallado

SET @ErrorMsg = ERROR_MESSAGE() +

' (Error: ' + CAST(ERROR_NUMBER() AS VARCHAR) +

', Nivel: ' + CAST(ERROR_SEVERITY() AS VARCHAR) +

```

```
    ', Línea: ' + CAST(ERROR_LINE() AS VARCHAR) + ');
```

```
-- 12. Registrar error en tabla de auditoría (opcional)
```

```
IF OBJECT_ID('Administracion.LogErrores', 'U') IS NULL
```

```
BEGIN
```

```
    CREATE TABLE Administracion.LogErrores (
```

```
        Id INT IDENTITY(1,1) PRIMARY KEY,
```

```
        Fecha DATETIME2 DEFAULT SYSDATETIME(),
```

```
        Procedimiento NVARCHAR(128),
```

```
        ErrorMsg NVARCHAR(4000),
```

```
        Parametros XML
```

```
    );
```

```
END
```

```
INSERT INTO Administracion.LogErrores (Procedimiento, ErrorMsg,  
Parametros)
```

```
VALUES (
```

```
    'SP_AprobarTituloProfesional',
```

```
    @ErrorMsg,
```

```
    (SELECT
```

```
        @IdTramite AS IdTramite,
```

```
        @NroResolucion AS NroResolucion,
```

```
        @FechaEmision AS FechaEmision,
```

```
        @OrganoEmisor AS OrganoEmisor,
```

```
        @NroDiploma AS NroDiploma
```

```
    FOR XML AUTO)
```

```
);
```

```
-- 13. Lanzar el error hacia la aplicación
```

```
THROW;
```

```
END CATCH
```

```
END
```

```
GO
```

```
-- 3. PROCEDIMIENTO PARA REVISAR ESTADO DE UN TRÁMITE ANTES  
DE APROBAR
```

```
CREATE OR ALTER PROCEDURE Administracion.SP_VerificarAprobacionTitulo
```

```
    @IdTramite INT
```

```
AS
```

```
BEGIN
```

```
    SET NOCOUNT ON;
```

```
    SELECT
```

```
        t.IdTramite,
```

```
        t.IdMatricula,
```

```
        m.IdEstudiante,
```

```
        p.DNI,
```

```
        p.Nombres + ' ' + p.ApellidoPaterno + ' ' + ISNULL(p.ApellidoMaterno, '') AS  
NombreCompleto,
```

```
        t.IdTipoTramite,
```

```
        tt.Descripcion AS TipoTramite,
```

```
        t.IdEstadoTramite,
```



```

et.Descripcion AS Estado,

CASE WHEN EXISTS (

    SELECT 1

    FROM Evaluacion.Sustentacion s

    JOIN Tramite.TrabajoInvestigacion ti ON ti.IdTrabajo = s.IdTrabajo

    WHERE ti.IdTramite = t.IdTramite

    AND s.Condicion = 'APROBADO'

) THEN 'APROBADO' ELSE 'PENDIENTE' END AS Sustentacion,

t.Observaciones,

t.FechaInicio,

t.FechaUltMod

FROM Tramite.Tramite t

JOIN Tramite.TipoTramite tt ON t.IdTipoTramite = tt.IdTipoTramite

JOIN Catalogo.EstadoTramite et ON t.IdEstadoTramite = et.IdEstadoTramite

JOIN Academico.Matricula m ON t.IdMatricula = m.IdMatricula

JOIN Academico.Persona p ON m.IdEstudiante = p.IdPersona

WHERE t.IdTramite = @IdTramite;

END

GO

```

-- 4. EJEMPLO DE USO DEL PROCEDIMIENTO

-- DATOS DE PRUEBA (Primero insertamos un trámite en estado APROBADO)

/*

-- Insertar datos de prueba (si no existen)

```

INSERT INTO Academico.Persona (DNI, Nombres, ApellidoPaterno,
ApellidoMaterno, Correo, TipoPersona)

```

```
VALUES ('12345678', 'Juan', 'Perez', 'Gomez', 'jperez@upla.edu.pe',  
'ESTUDIANTE');
```

```
INSERT INTO Academico.Matricula (IdEstudiante, IdPrograma, FechaMatricula,  
AsignaturasAprobadas, PracticasCompletadas, IdiomaAprobado, ProyeccionSocial,  
TieneDeuda)
```

```
VALUES (1, 1, GETDATE(), 1, 1, 1, 1, 0);
```

```
INSERT INTO Tramite.Tramite (IdMatricula, IdTipoTramite, IdEstadoTramite,  
IdCoordinacion)
```

```
VALUES (1, 2, 5, 1);
```

-- Ejecutar la aprobación

```
DECLARE @IdRes INT, @IdDip INT, @ErrMsg NVARCHAR(4000);
```

```
EXEC Administracion.SP_AprobarTituloProfesional
```

```
    @IdTramite = 1,
```

```
    @NroResolucion = 'RES-2026-001-CC',
```

```
    @FechaEmision = GETDATE(),
```

```
    @OrganoEmisor = 'CONSEJO_FACULTAD',
```

```
    @NroDiploma = 'DIP-2026-001',
```

```
    @FirmanteRector = 'Dr. Jorge Peña',
```

```
    @FirmanteDecano = 'Mg. Maria Flores',
```

```
    @FirmanteSecretario = 'Lic. Pedro Ruiz',
```

```
    @Observaciones = 'Aprobado con unanimidad',
```

```
    @IdResolucion = @IdRes OUTPUT,
```

```
    @IdDiploma = @IdDip OUTPUT,
```

```
    @ErrorMsg = @ErrMsg OUTPUT;
```

```
SELECT @IdRes AS IdResolucion, @IdDip AS IdDiploma, @ErrMsg AS Mensaje;
```

```
*/
```

```
-- 5. PROCEDIMIENTO PARA ACTUALIZAR ESTADO DE UN TRÁMITE CON  
BLOQUEO (NOLOCK vs READPAST)
```

```
-- Esta función muestra cómo obtener datos sin bloquear otras operaciones
```

```
CREATE OR ALTER FUNCTION Administracion.F_TramiteLeve()
```

```
RETURNS TABLE
```

```
AS
```

```
RETURN
```

```
(
```

```
    SELECT
```

```
        t.IdTramite,
```

```
        t.IdMatricula,
```

```
        t.IdTipoTramite,
```

```
        t.IdEstadoTramite,
```

```
        t.FechaInicio,
```

```
        t.Observaciones
```

```
    FROM Tramite.Tramite t WITH (NOLOCK) -- Lectura sucia permitida para  
reportes
```

```
    WHERE t.IdEstadoTramite IN (1, 2, 3) -- Estados activos
```

```
);
```

```
GO
```

```
-- 6. PROCEDIMIENTO CON TRANSACCIÓN PARAMETRIZABLE  
(AISLAMIENTO)
```

```
CREATE OR ALTER PROCEDURE
Administracion.SP_ActualizarEstadoTramiteConIsolation
```

```
    @IdTramite INT,
```

```
    @NuevoEstado TINYINT,
```

```
    @IsolationLevel VARCHAR(20) = 'READ COMMITTED'
```

```
AS
```

```
BEGIN
```

```
    SET NOCOUNT ON;
```

```
    DECLARE @TranName VARCHAR(32) = 'Tran_ActualizacionEstado';
```

```
    -- Establecer nivel de aislamiento según parámetro
```

```
    SET @IsolationLevel = UPPER(@IsolationLevel);
```

```
    IF @IsolationLevel = 'READ UNCOMMITTED'
```

```
        SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
```

```
    ELSE IF @IsolationLevel = 'READ COMMITTED'
```

```
        SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

```
    ELSE IF @IsolationLevel = 'REPEATABLE READ'
```

```
        SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

```
    ELSE IF @IsolationLevel = 'SERIALIZABLE'
```

```
        SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

```
    ELSE IF @IsolationLevel = 'SNAPSHOT'
```

```
        SET TRANSACTION ISOLATION LEVEL SNAPSHOT;
```

```
    ELSE
```

```
        THROW 50005, 'Nivel de aislamiento no válido. Opciones: READ
UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE,
SNAPSHOT', 1;
```

```

BEGIN TRY

    BEGIN TRANSACTION @TranName;

    -- Validar que el estado existe

    IF NOT EXISTS (SELECT 1 FROM Catalogo.EstadoTramite WHERE
    IdEstadoTramite = @NuevoEstado)

        THROW 50006, 'El estado especificado no existe en el catálogo.', 1;

    UPDATE Tramite.Tramite

    SET IdEstadoTramite = @NuevoEstado,

        FechaUltMod = SYSDATETIME()

    WHERE IdTramite = @IdTramite;

    COMMIT TRANSACTION @TranName;

    PRINT 'Estado actualizado exitosamente para el trámite ' + CAST(@IdTramite
    AS VARCHAR);

END TRY

BEGIN CATCH

    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION @TranName;

    THROW;

END CATCH

END

GO

```

Proyecto 2: Simulación y resolución de bloqueos Enunciado Dos coordinadores modifican simultáneamente el mismo trámite provocando bloqueos prolongados.

Codigo:

```
--  
=====
```

```
-- PROYECTO 2: Simulación y resolución de bloqueos
```

```
-- Dos coordinadores modifican el mismo trámite
```

```
--  
=====
```

USE GradosTitulos;

GO

```
-- 1. CREAR TABLA PARA REGISTRO DE BLOQUEOS
```

```
IF OBJECT_ID('Administracion.LogBloqueos', 'U') IS NULL
```

```
BEGIN
```

```
    CREATE TABLE Administracion.LogBloqueos (
```

```
        Id INT IDENTITY(1,1) PRIMARY KEY,
```

```
        Fecha DATETIME2 DEFAULT SYSDATETIME(),
```

```
        Tipo VARCHAR(50),
```

```
        Descripcion NVARCHAR(4000),
```

```
        DatosXML XML
```

```
    );
```

```
END
```

GO

-- 2. PROCEDIMIENTO SIMULADO PARA COORDINADOR 1 (CON BLOQUEO)

CREATE OR ALTER PROCEDURE

Administracion.SP_Coordinador1_ActualizarTramite

@IdTramite INT,

@NuevoEstado TINYINT,

@Observacion NVARCHAR(500)

AS

BEGIN

SET NOCOUNT ON;

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

DECLARE @TranName VARCHAR(32) = 'Tran_Coordinador1';

BEGIN TRY

-- Simular un proceso que toma tiempo

WAITFOR DELAY '00:00:05'; -- 5 segundos de espera

BEGIN TRANSACTION @TranName;

-- Leer el registro actual con bloqueo (SELECT con XLOCK)

SELECT IdTramite, IdEstadoTramite, Observaciones

FROM Tramite.Tramite WITH (XLOCK, ROWLOCK)

WHERE IdTramite = @IdTramite;

-- Simular procesamiento

WAITFOR DELAY '00:00:03';

```

-- Actualizar el trámite

UPDATE Trámite.Trámite

SET

    IdEstadoTrámite = @NuevoEstado,

    Observaciones = COALESCE(Observaciones, '') +

        CASE WHEN Observaciones IS NOT NULL THEN '|' ELSE '' END +

        'COORD1: ' + @Observacion,

    FechaUltMod = SYSDATETIME()

WHERE IdTrámite = @IdTrámite;

COMMIT TRANSACTION @TranName;

PRINT 'Coordinador 1: Trámite ' + CAST(@IdTrámite AS VARCHAR) + '
actualizado.';

END TRY

BEGIN CATCH

    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION @TranName;

-- Registrar el error

INSERT INTO Administracion.LogBloqueos (Tipo, Descripción, DatosXML)

VALUES (

    'BLOQUEO',

    'Error en Coordinador1 para trámite ' + CAST(@IdTrámite AS VARCHAR) +
': ' + ERROR_MESSAGE(),

    (SELECT

        @IdTrámite AS IdTrámite,

```



```

        @NuevoEstado AS NuevoEstado,

        @Observacion AS Observacion

    FOR XML AUTO)

);

    THROW;

END CATCH

END

GO

-- 3. PROCEDIMIENTO SIMULADO PARA COORDINADOR 2 (CON
BLOQUEO)

CREATE OR ALTER PROCEDURE
Administracion.SP_Coordinador2_ActualizarTramite

    @IdTramite INT,

    @NuevoEstado TINYINT,

    @Observacion NVARCHAR(500)

AS

BEGIN

    SET NOCOUNT ON;

    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

    DECLARE @TranName VARCHAR(32) = 'Tran_Coordinador2';

    BEGIN TRY

        -- Simular un proceso que toma tiempo (invertido para generar bloqueo)

        WAITFOR DELAY '00:00:02';

```

```

BEGIN TRANSACTION @TranName;

-- Intentar leer y actualizar

SELECT IdTramite, IdEstadoTramite, Observaciones
FROM Tramite.Tramite WITH (XLOCK, ROWLOCK)
WHERE IdTramite = @IdTramite;

-- Simular procesamiento más largo

WAITFOR DELAY '00:00:06';

UPDATE Tramite.Tramite
SET
    IdEstadoTramite = @NuevoEstado,
    Observaciones = COALESCE(Observaciones, '') +
        CASE WHEN Observaciones IS NOT NULL THEN '|' ELSE '' END +
        'COORD2: ' + @Observacion,
    FechaUltMod = SYSDATETIME()
WHERE IdTramite = @IdTramite;

COMMIT TRANSACTION @TranName;

PRINT 'Coordinador 2: Tramite ' + CAST(@IdTramite AS VARCHAR) + '
actualizado.';

END TRY

BEGIN CATCH

    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION @TranName;

```

```

INSERT INTO Administracion.LogBloqueos (Tipo, Descripcion, DatosXML)
VALUES (
    'BLOQUEO',
    'Error en Coordinador2 para trámite ' + CAST(@IdTramite AS VARCHAR) +
    ': ' + ERROR_MESSAGE(),
    (SELECT
        @IdTramite AS IdTramite,
        @NuevoEstado AS NuevoEstado,
        @Observacion AS Observacion
    FOR XML AUTO)
);

THROW;

END CATCH

END

GO

```

-- 4. PROCEDIMIENTO PARA DETECTAR BLOQUEOS ACTIVOS (DMV)

```
CREATE OR ALTER PROCEDURE Administracion.SP_DetectarBloqueos
```

```
AS
```

```
BEGIN
```

```
    SET NOCOUNT ON;
```

-- 4.1 Bloqueos actuales (esperas)

```
SELECT
```

```

blocking.session_id AS Bloqueador_SessionID,

blocking.login_name AS Bloqueador_Login,

blocking.status AS Bloqueador_Estado,

blocked.session_id AS Bloqueado_SessionID,

blocked.login_name AS Bloqueado_Login,

blocked.status AS Bloqueado_Estado,

blocked.wait_type AS Tipo_Espera,

blocked.wait_time AS Tiempo_Espera_ms,

blocked.last_wait_type AS Ultimo_Tipo_Espera,

blocked.wait_resource AS Recurso,

blocked.command AS Comando,

blocked.text AS TextoSQL,

DATEDIFF(SECOND, blocked.last_wait_started, GETDATE()) AS
Segundos_Esperando

FROM sys.dm_exec_requests blocked

LEFT JOIN sys.dm_exec_sessions blocking ON blocked.blocking_session_id =
blocking.session_id

CROSS APPLY sys.dm_exec_sql_text(blocked.sql_handle) AS text

WHERE blocked.blocking_session_id > 0;

```

-- 4.2 Resumen de bloqueos por objeto

```

SELECT

DB_NAME(resource_database_id) AS BaseDatos,

OBJECT_NAME(resource_associated_entity_id) AS Objeto,

resource_type AS TipoRecurso,

resource_description AS Descripcion,

```

```

COUNT(*) AS CantidadBloqueos,

MAX(request_owner_id) AS UltimoBloqueo

FROM sys.dm_tran_locks

WHERE resource_database_id = DB_ID('GradosTitulos')

AND request_status = 'WAIT'

AND resource_type IN ('PAGE', 'KEY', 'RID', 'OBJECT')

GROUP BY resource_database_id, resource_associated_entity_id, resource_type,
resource_description

ORDER BY CantidadBloqueos DESC;

END

GO

```

-- 5. PROCEDIMIENTO PARA MATAR BLOQUEOS (CON PRECAUCIÓN)

```

CREATE OR ALTER PROCEDURE Administracion.SP_ResolverBloqueo

```

```

    @SessionID INT,

```

```

    @KillWithStatusOnly BIT = 0 -- 0 = KILL, 1 = STATUS ONLY

```

```

AS

```

```

BEGIN

```

```

    SET NOCOUNT ON;

```

```

    IF @KillWithStatusOnly = 1

```

```

    BEGIN

```

```

        -- Solo mostrar estado

```

```

        SELECT

```

```

            session_id,

```

```

            login_name,

```

```

        status,

        command,

        wait_type,

        wait_time,

        wait_resource,

        blocking_session_id

FROM sys.dm_exec_requests

WHERE session_id = @SessionID;

END

ELSE

BEGIN

    -- Intentar matar la sesión

    DECLARE @Command NVARCHAR(50) = 'KILL ' + CAST(@SessionID AS
    VARCHAR);

    BEGIN TRY

        EXEC sp_executesql @Command;

        PRINT 'Sesión ' + CAST(@SessionID AS VARCHAR) + ' finalizada.';

        INSERT INTO Administracion.LogBloqueos (Tipo, Descripcion)

        VALUES ('KILL', 'Sesión ' + CAST(@SessionID AS VARCHAR) + '
        finalizada manualmente.');
```

```

    END TRY

    BEGIN CATCH

        PRINT 'Error al finalizar sesión: ' + ERROR_MESSAGE();

        INSERT INTO Administracion.LogBloqueos (Tipo, Descripcion)
```

```
VALUES ('KILL_ERROR', 'Error al finalizar sesión ' + CAST(@SessionID
AS VARCHAR) + ': ' + ERROR_MESSAGE());
```

```
END CATCH
```

```
END
```

```
END
```

```
GO
```

-- 6. PROCEDIMIENTO PARA SIMULAR BLOQUEO COMPLETO (DOS SESIONES)

-- Ejecutar esta simulación en DOS ventanas de SSMS diferentes

-- Ventana 1: Ejecutar COORDINADOR 1

-- Ventana 2: Ejecutar COORDINADOR 2 (aproximadamente al mismo tiempo)

```
/*
```

-- VENTANA 1 (Coordinador 1)

```
EXEC Administracion.SP_Coordinador1_ActualizarTramite
```

```
@IdTramite = 1,
```

```
@NuevoEstado = 4,
```

```
@Observacion = 'En espera de documentación';
```

```
GO
```

-- VENTANA 2 (Coordinador 2) - Ejecutar casi simultáneamente

```
EXEC Administracion.SP_Coordinador2_ActualizarTramite
```

```
@IdTramite = 1,
```

```
@NuevoEstado = 3,
```

```
@Observacion = 'Revisión de documentos';
```

GO

*/

-- 7. VERSIÓN MEJORADA CON MANEJO DE DEADLOCK (RETRY)

CREATE OR ALTER PROCEDURE

Administracion.SP_ActualizarTramiteConRetry

 @IdTramite INT,

 @NuevoEstado TINYINT,

 @Observacion NVARCHAR(500),

 @MaxRetries INT = 3

AS

BEGIN

 SET NOCOUNT ON;

 DECLARE @RetryCount INT = 0;

 DECLARE @TranName VARCHAR(32) = 'Tran_Retry';

 DECLARE @ErrorMsg NVARCHAR(4000);

 WHILE @RetryCount < @MaxRetries

 BEGIN

 BEGIN TRY

 SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

 BEGIN TRANSACTION @TranName;

 -- Verificar que el trámite existe y no está bloqueado


```
IF NOT EXISTS (SELECT 1 FROM Tramite.Tramite WHERE IdTramite =  
@IdTramite)
```

```
    THROW 50007, 'El trámite especificado no existe.', 1;
```

```
-- Actualizar con bloqueo controlado
```

```
UPDATE Tramite.Tramite
```

```
SET
```

```
    IdEstadoTramite = @NuevoEstado,
```

```
    Observaciones = COALESCE(Observaciones, '') +
```

```
        CASE WHEN Observaciones IS NOT NULL THEN '|' ELSE '' END +
```

```
        'RETRY-' + CAST(@RetryCount + 1 AS VARCHAR) + ': ' +  
@Observacion,
```

```
    FechaUltMod = SYSDATETIME()
```

```
WHERE IdTramite = @IdTramite;
```

```
COMMIT TRANSACTION @TranName;
```

```
PRINT 'Actualización exitosa en intento ' + CAST(@RetryCount + 1 AS  
VARCHAR) + ' para trámite ' + CAST(@IdTramite AS VARCHAR);
```

```
RETURN 0; -- Éxito
```

```
END TRY
```

```
BEGIN CATCH
```

```
IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION @TranName;
```

```
-- Verificar si es un deadlock (error 1205) o bloqueo (error 1222)
```

```
IF ERROR_NUMBER() IN (1205, 1222)
```

```
BEGIN
```

```

        SET @RetryCount = @RetryCount + 1;

        SET @ErrorMsg = 'Intento ' + CAST(@RetryCount AS VARCHAR) + '
falló: ' + ERROR_MESSAGE();

        PRINT @ErrorMsg;

        -- Esperar antes de reintentar (backoff exponencial)

        WAITFOR DELAY '00:00:0' + CAST(@RetryCount AS VARCHAR) +
'.000';

    END

    ELSE

    BEGIN

        -- Otro tipo de error, no reintentar

        THROW;

    END

    END CATCH

END

-- Si llegamos aquí, todos los intentos fallaron

    SET @ErrorMsg = 'No se pudo actualizar el trámite después de ' +
CAST(@MaxRetries AS VARCHAR) + ' intentos.';

    PRINT @ErrorMsg;

    INSERT INTO Administracion.LogBloqueos (Tipo, Descripcion)

    VALUES ('RETRY_FAILED', @ErrorMsg);

    RETURN -1; -- Fracaso

END

```

GO

-- 8. SCRIPT DE MONITOREO CONTINUO DE BLOQUEOS

-- Ejecutar en una ventana separada para monitorear en tiempo real

CREATE OR ALTER PROCEDURE
Administracion.SP_MonitoreoBloqueosEnVivo

AS

BEGIN

SET NOCOUNT ON;

PRINT '=== MONITOREO DE BLOQUEOS EN TIEMPO REAL ===';

PRINT 'Presiona Ctrl+C para detener';

PRINT '=====';

DECLARE @Counter INT = 0;

WHILE 1 = 1

BEGIN

SET @Counter = @Counter + 1;

-- Limpiar pantalla (solo en SSMS usando PRINT)

PRINT REPLICATE('-', 80);

PRINT 'Iteración ' + CAST(@Counter AS VARCHAR) + ' - ' +
CONVERT(VARCHAR, GETDATE(), 120);

PRINT REPLICATE('-', 80);

```

-- Mostrar bloqueos activos

EXEC Administracion.SP_DetectarBloqueos;


-- Esperar 5 segundos

WAITFOR DELAY '00:00:05';

END

END

GO


-- 9. CREAR VISTA PARA BLOQUEOS DE TRÁMITES (Fácil consulta)

CREATE OR ALTER VIEW Administracion.V_BloqueosTramites

AS

WITH BloqueosCTE AS (

    SELECT

        blocked.session_id AS SessionID_Bloqueado,

        blocked.login_name AS Login_Bloqueado,

        blocking.session_id AS SessionID_Bloqueador,

        blocking.login_name AS Login_Bloqueador,

        blocked.wait_type AS TipoEspera,

        blocked.wait_time AS TiempoEspera,

        blocked.wait_resource AS Recurso,

        blocked.text AS TextoSQL_Bloqueado,

        DATEDIFF(SECOND, blocked.last_wait_started, GETDATE()) AS
SegundosEsperando

    FROM sys.dm_exec_requests blocked

```

```
    LEFT JOIN sys.dm_exec_sessions blocking ON blocked.blocking_session_id =  
blocking.session_id
```

```
    CROSS APPLY sys.dm_exec_sql_text(blocked.sql_handle)
```

```
    WHERE blocked.blocking_session_id > 0
```

```
)
```

```
SELECT * FROM BloqueosCTE;
```

```
GO
```

-- 10. VERIFICAR EL ESTADO DE BLOQUEOS DESPUÉS DE LA
SIMULACIÓN

```
EXEC Administracion.SP_DetectarBloqueos;
```

```
GO
```

-- Consultar logs de bloqueos

```
SELECT * FROM Administracion.LogBloqueos ORDER BY Fecha DESC;
```

```
GO
```

-- Ver bloqueos activos en la vista

```
SELECT * FROM Administracion.V_BloqueosTramites;
```

```
GO
```

TEMA 4. Análisis de planes de ejecución

Proyecto 1: Diagnóstico de consultas complejas Enunciado Las consultas que generan reportes institucionales presentan tiempos superiores a 10 segundos.

Codigo:--

```
=====
```

-- PROYECTO 1: Diagnóstico de consultas complejas

-- Identificar consultas > 10 segundos en GradosTitulos

--

```
=====
```

USE GradosTitulos;

GO

-- 1. CREAR ESQUEMA ADMINISTRACION SI NO EXISTE

IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'Administracion')

EXEC ('CREATE SCHEMA Administracion');

GO

-- 2. TABLA PARA ALMACENAR CONSULTAS LENTAS HISTÓRICAS

IF OBJECT_ID('Administracion.ConsultasLentasHistoricas', 'U') IS NULL

BEGIN

CREATE TABLE Administracion.ConsultasLentasHistoricas (

Id INT IDENTITY(1,1) PRIMARY KEY,

FechaCaptura DATETIME2 DEFAULT SYSDATETIME(),

QueryText NVARCHAR(MAX),

QueryPlan XML,

DuracionMs BIGINT,

```

    CpuTimeMs BIGINT,

    LogicalReads BIGINT,

    PhysicalReads BIGINT,

    Writes BIGINT,

    RowCount BIGINT,

    DatabaseName NVARCHAR(128),

    LoginName NVARCHAR(128),

    HostName NVARCHAR(128),

    ProgramName NVARCHAR(128),

    IsSystem BIT DEFAULT 0,

    FechaProcesado DATETIME2 NULL,

    Observaciones NVARCHAR(500) NULL

);

END

GO

-- 3. PROCEDIMIENTO PARA CAPTURAR CONSULTAS LENTAS ACTIVAS

CREATE OR ALTER PROCEDURE Administracion.SP_CapturarConsultasLentas

    @UmbralMs INT = 10000, -- 10 segundos por defecto

    @TopN INT = 50

AS

BEGIN

    SET NOCOUNT ON;

    INSERT INTO Administracion.ConsultasLentasHistoricas (

```

QueryText,
QueryPlan,
DuracionMs,
CpuTimeMs,
LogicalReads,
PhysicalReads,
Writes,
RowCount,
DatabaseName,
LoginName,
HostName,
ProgramName,
IsSystem,
FechaProcesado
)

SELECT TOP (@TopN)

TEXT.text AS QueryText,
plan.query_plan AS QueryPlan,
qs.total_elapsed_time / 1000 AS DuracionMs,
qs.total_worker_time / 1000 AS CpuTimeMs,
qs.total_logical_reads AS LogicalReads,
qs.total_physical_reads AS PhysicalReads,
qs.total_writes AS Writes,
qs.total_rows AS RowCount,
DB_NAME(qt.dbid) AS DatabaseName,


```

    sess.login_name AS LoginName,

    sess.host_name AS HostName,

    sess.program_name AS ProgramName,

    CASE WHEN sess.is_user_process = 0 THEN 1 ELSE 0 END AS IsSystem,

    NULL AS FechaProcesado

FROM sys.dm_exec_query_stats qs

CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) AS TEXT

CROSS APPLY sys.dm_exec_query_plan(qs.plan_handle) AS plan

LEFT JOIN sys.dm_exec_sessions sess ON qs.session_id = sess.session_id

WHERE (qs.total_elapsed_time / 1000) > @UmbralMs

    AND DB_NAME(qt.dbid) = 'GradosTitulos'

    AND TEXT.text NOT LIKE '%ConsultasLentasHistoricas%' -- Excluir el mismo
proceso

    AND TEXT.text NOT LIKE '%SP_CapturarConsultasLentas%'

ORDER BY qs.total_elapsed_time DESC;

PRINT 'Se capturaron ' + CAST(@@ROWCOUNT AS VARCHAR) + ' consultas
lentas.';

END

GO

```

-- 4. PROCEDIMIENTO PARA ANALIZAR PLAN DE EJECUCIÓN DETALLADO

```

CREATE OR ALTER PROCEDURE Administracion.SP_AnalizarPlanEjecucion

    @IdConsulta INT,

    @AnalisisDetallado BIT = 1

AS

```

BEGIN

SET NOCOUNT ON;

DECLARE @QueryText NVARCHAR(MAX), @QueryPlan XML, @DuracionMs
BIGINT, @LogicalReads BIGINT;

-- Obtener datos de la consulta

SELECT

@QueryText = QueryText,

@QueryPlan = QueryPlan,

@DuracionMs = DuracionMs,

@LogicalReads = LogicalReads

FROM Administracion.ConsultasLentasHistoricas

WHERE Id = @IdConsulta;

IF @QueryText IS NULL

BEGIN

PRINT 'No se encontró la consulta con ID ' + CAST(@IdConsulta AS
VARCHAR);

RETURN;

END

-- Mostrar información general

PRINT '=====';

PRINT 'ANÁLISIS DE PLAN DE EJECUCIÓN - CONSULTA ID: ' +
CAST(@IdConsulta AS VARCHAR);

PRINT '=====';

```

PRINT 'Duración: ' + CAST(@DuracionMs AS VARCHAR) + ' ms';

PRINT 'Lecturas Lógicas: ' + CAST(@LogicalReads AS VARCHAR);

PRINT '-----';

PRINT 'Texto de la consulta (primeros 500 caracteres):';

PRINT LEFT(@QueryText, 500) + '...';

PRINT '=====';

IF @AnalisisDetallado = 1 AND @QueryPlan IS NOT NULL
BEGIN

    -- Análisis del plan: identificar operadores costosos

    PRINT ";

    PRINT 'ANÁLISIS DE OPERADORES COSTOSOS:';

    PRINT '-----';


    -- Extraer operadores del plan

    WITH PlanOperators AS (

        SELECT

            node.value('@StatementText', 'NVARCHAR(MAX)') AS StatementText,

            node.value('@StatementType', 'VARCHAR(50)') AS StatementType,

            node.value('@StatementSubTreeCost', 'DECIMAL(20,6)') AS
StatementSubTreeCost,

            node.value('@MaxDOP', 'INT') AS MaxDOP,

            node.query('.') AS OperatorXML

        FROM @QueryPlan.nodes('//StmtSimple') AS nodes(node)

    )

    SELECT

```

```

StatementType,

StatementSubTreeCost,

LEFT(StatementText, 200) AS StatementText

FROM PlanOperators

ORDER BY StatementSubTreeCost DESC;


-- Buscar Table Scan o Clustered Index Scan (más costosos)

PRINT ";

PRINT 'OPERADORES DE ESCANEEO IDENTIFICADOS:';

PRINT '-----';


WITH Scans AS (

    SELECT

        node.value('@PhysicalOp', 'VARCHAR(50)') AS PhysicalOp,

        node.value('@LogicalOp', 'VARCHAR(50)') AS LogicalOp,

        node.value('@EstimatedCost', 'DECIMAL(20,6)') AS EstimatedCost,

        node.value('@EstimatedRows', 'BIGINT') AS EstimatedRows,

        node.value('@Table', 'NVARCHAR(200)') AS TableName

    FROM @QueryPlan.nodes('//RelOp') AS nodes(node)

    WHERE node.value('@PhysicalOp', 'VARCHAR(50)') IN (

        'Table Scan', 'Clustered Index Scan', 'Index Scan',

        'Table Seek', 'Clustered Index Seek', 'Index Seek'

    )

)

SELECT

```

```

PhysicalOp,

LogicalOp,

EstimatedCost,

EstimatedRows,

TableName

FROM Scans

ORDER BY EstimatedCost DESC;

END

-- Recomendaciones básicas basadas en el análisis

PRINT ";

PRINT 'RECOMENDACIONES PRELIMINARES:.';

PRINT '-----';

IF @LogicalReads > 100000

    PRINT '⚠ Alto número de lecturas lógicas (>100K). Considere:.';

ELSE IF @LogicalReads > 50000

    PRINT '⚠ Número moderado de lecturas lógicas (>50K). Considere:.';

IF @LogicalReads > 50000

BEGIN

    PRINT ' • Crear índices cubrientes para las columnas más consultadas.';

    PRINT ' • Revisar filtros WHERE para usar columnas indexadas.';

    PRINT ' • Considerar vistas materializadas para reportes frecuentes.';

END

```

```

IF @DuracionMs > 30000

    PRINT '⚠ Consulta extremadamente lenta (>30 segundos). Recomendación
prioritaria!';

PRINT ' • Ejecutar UPDATE STATISTICS con FULLSCAN!';

PRINT ' • Revisar el plan de ejecución para identificar operadores de escaneo!';

PRINT ' • Considerar particionar tablas grandes!';


-- Marcar como analizada

UPDATE Administracion.ConsultasLentasHistoricas

SET FechaProcesado = SYSDATETIME(),

    Observaciones = 'Analizado por SP_AnalizarPlanEjecucion'

WHERE Id = @IdConsulta;

END

GO


-- 5. CONSULTA DE REPORTE DE GRADOS Y TÍTULOS (SIMULACIÓN DE
CONSULTA COMPLEJA)

-- Esta consulta simula un reporte institucional lento


CREATE OR ALTER VIEW Administracion.V_ReporteGradosTitulos

AS

WITH DatosTramites AS (

    SELECT

        t.IdTramite,

```

t.IdMatricula,
t.IdTipoTramite,
tt.Descripcion AS TipoTramite,
t.IdEstadoTramite,
et.Descripcion AS Estado,
t.FechaInicio,
t.FechaUltMod,
t.Observaciones,
m.IdEstudiante,
p.DNI,
p.Nombres,
p.ApellidoPaterno,
p.ApellidoMaterno,
p.Correo,
m.IdPrograma,
pr.NombrePrograma,
pr.NumSemestres,
pr.Modalidad,
f.NombreFacultad,
f.NombreDecano,
-- Información de trabajo de investigación
ti.IdTrabajo,
ti.TipoTrabajo,
ti.Titulo AS TituloTrabajo,
ti.ModalidadAutoria,

```

ti.PorcentajeSimilitud,
ti.UrlRepositorio,
-- Información de sustentación
s.IdSustentacion,
s.FechaHora AS FechaSustentacion,
s.NotaExposicion,
s.NotaDefensa,
s.Condicion AS CondicionSustentacion,
s.NroActa,
-- Información de diplomas
d.IdDiploma,
d.NroDiploma,
d.FechaExpedicion,
d.FechaRegistroSunedu,
-- Información de resoluciones
r.IdResolucion,
r.NroResolucion,
r.FechaEmision,
r.TipoResolucion,
r.OrganoEmisor,
-- Información de pagos
COUNT(pg.IdPago) AS NumPagos,
SUM(pg.Monto) AS TotalPagado
FROM Tramite.Tramite t
LEFT JOIN Tramite.TipoTramite tt ON t.IdTipoTramite = tt.IdTipoTramite

```


LEFT JOIN Catalogo.EstadoTramite et ON t.IdEstadoTramite =
et.IdEstadoTramite

LEFT JOIN Academico.Matricula m ON t.IdMatricula = m.IdMatricula

LEFT JOIN Academico.Persona p ON m.IdEstudiante = p.IdPersona

LEFT JOIN Academico.ProgramaEstudios pr ON m.IdPrograma = pr.IdPrograma

LEFT JOIN Academico.Facultad f ON pr.IdFacultad = f.IdFacultad

LEFT JOIN Tramite.TrabajoInvestigacion ti ON t.IdTramite = ti.IdTramite

LEFT JOIN Evaluacion.Sustentacion s ON ti.IdTrabajo = s.IdTrabajo AND
s.NroIntento = 1

LEFT JOIN Documento.Diploma d ON t.IdTramite = d.IdTramite

LEFT JOIN Documento.Resolucion r ON t.IdTramite = r.IdTramite AND
r.TipoResolucion = 'APROBACION_TITULO'

LEFT JOIN Tramite.Pago pg ON t.IdTramite = pg.IdTramite

GROUP BY

t.IdTramite, t.IdMatricula, t.IdTipoTramite, tt.Descripcion,
t.IdEstadoTramite, et.Descripcion, t.FechaInicio, t.FechaUltMod,
t.Observaciones, m.IdEstudiante, p.DNI, p.Nombres, p.ApellidoPaterno,
p.ApellidoMaterno, p.Correo, m.IdPrograma, pr.NombrePrograma,
pr.NumSemestres, pr.Modalidad, f.NombreFacultad, f.NombreDecano,
ti.IdTrabajo, ti.TipoTrabajo, ti.Titulo, ti.ModalidadAutoria,
ti.PorcentajeSimilitud, ti.UrlRepositorio, s.IdSustentacion,
s.FechaHora, s.NotaExposicion, s.NotaDefensa, s.Condicion,
s.NroActa, d.IdDiploma, d.NroDiploma, d.FechaExpedicion,
d.FechaRegistroSunedu, r.IdResolucion, r.NroResolucion,
r.FechaEmision, r.TipoResolucion, r.OrganoEmisor

)

SELECT * FROM DatosTramites;

GO

-- 6. EJECUTAR CONSULTA COMPLEJA PARA SIMULAR RENDIMIENTO LENTO

-- Esta consulta simula un reporte que genera lentitud

/*

-- Ejecutar esta consulta para generar tráfico y recolectar métricas

SELECT

IdTramite,

NombreFacultad,

NombrePrograma,

TipoTramite,

Estado,

FechaInicio,

FechaSustentacion,

CondicionSustentacion,

TotalPagado,

COUNT(*) OVER (PARTITION BY NombreFacultad) AS TotalPorFacultad,

AVG(DATEDIFF(DAY, FechaInicio, FechaSustentacion)) OVER () AS
PromedioDiasAprobacion

FROM Administracion.V_ReporteGradosTitulos

WHERE Estado IN ('APROBADO', 'ARCHIVADO')

AND FechaInicio >= DATEADD(MONTH, -6, GETDATE())

AND NombreFacultad IS NOT NULL

ORDER BY NombreFacultad, FechaInicio DESC;

GO

*/

-- 7. CAPTURAR Y ANALIZAR CONSULTAS LENTAS

EXEC Administracion.SP_CapturarConsultasLentas @UmbralMs = 10000, @TopN
= 20;

GO

-- Ver consultas capturadas

SELECT

Id,

FechaCaptura,

LEFT(QueryText, 150) AS QueryResumida,

DuracionMs,

CpuTimeMs,

LogicalReads,

LoginName

FROM Administracion.ConsultasLentasHistoricas

ORDER BY DuracionMs DESC;

GO

-- Analizar una consulta específica (reemplazar @IdConsulta con un ID real)

-- EXEC Administracion.SP_AnalizarPlanEjecucion @IdConsulta = 1,
@AnalisisDetallado = 1;

-- GO

Proyecto 2: Comparación de planes antes y después de crear índices Enunciado

La Dirección de Informática desea comprobar el impacto real de nuevos índices sobre las consultas del módulo de grados.

Codigo:

```
--  
=====
```

```
-- PROYECTO 2: Comparación de planes antes y después de índices
```

```
-- Medir impacto real de índices en consultas clave
```

```
--  
=====
```

USE GradosTitulos;

GO

```
-- 1. CREAR TABLA PARA ALMACENAR PLANES DE EJECUCIÓN  
HISTÓRICOS
```

```
IF OBJECT_ID('Administracion.PlanesEjecucionHistoricos', 'U') IS NULL
```

```
BEGIN
```

```
    CREATE TABLE Administracion.PlanesEjecucionHistoricos (
```

```
        Id INT IDENTITY(1,1) PRIMARY KEY,
```

```
        FechaCaptura DATETIME2 DEFAULT SYSDATETIME(),
```

```
        QueryText NVARCHAR(MAX),
```

```
        QueryPlan XML,
```

```
        Estadisticas NVARCHAR(MAX),
```

```
        DuracionMs BIGINT,
```

```
        CpuTimeMs BIGINT,
```

```
        LogicalReads BIGINT,
```

```
        PhysicalReads BIGINT,
```

```
Writes BIGINT,  
  
RowCount BIGINT,  
  
Escenario NVARCHAR(50), -- 'ANTES_INDICE' o 'DESPUES_INDICE'  
  
IndicesCreados NVARCHAR(MAX),  
  
EsPlanCompilado BIT DEFAULT 1  
  
);  
  
END  
  
GO
```

-- 2. PROCEDIMIENTO PARA CAPTURAR PLAN DE EJECUCIÓN CON ESTADÍSTICAS

```
CREATE OR ALTER PROCEDURE Administracion.SP_CapturarPlanEjecucion
```

```
    @QueryText NVARCHAR(MAX),  
  
    @Escenario NVARCHAR(50),  
  
    @IndicesCreados NVARCHAR(MAX) = NULL,  
  
    @Parametros NVARCHAR(MAX) = NULL
```

```
AS
```

```
BEGIN
```

```
    SET NOCOUNT ON;  
  
    SET STATISTICS XML ON;  
  
    SET STATISTICS TIME ON;  
  
    SET STATISTICS IO ON;
```

```
    DECLARE @StartTime DATETIME2 = SYSDATETIME();
```

```
    DECLARE @PlanXML XML;
```

```
    DECLARE @StatsOutput NVARCHAR(MAX) = '';
```

```

BEGIN TRY

    -- Ejecutar la consulta con dinámico

    DECLARE @Sql NVARCHAR(MAX);

    IF @Parametros IS NOT NULL

        SET @Sql = 'EXEC sp_executesql N''' + @QueryText + ''', N''' +
@Parametros + ''';

    ELSE

        SET @Sql = @QueryText;

    -- Capturar estadísticas de tiempo

    PRINT 'Ejecutando consulta...';

    EXEC sp_executesql @Sql;

    -- Guardar el plan y estadísticas

    SELECT @PlanXML = CONVERT(XML, '<?xml version="1.0" encoding="utf-
16"?>' +

        SUBSTRING((SELECT query_plan FROM sys.dm_exec_query_stats

        CROSS APPLY sys.dm_exec_sql_text(sql_handle)

        CROSS APPLY sys.dm_exec_query_plan(plan_handle)

        WHERE TEXT = @QueryText FOR XML PATH('')), 1, 8000));

    -- Insertar en tabla

    INSERT INTO Administracion.PlanesEjecucionHistoricos (

        QueryText,

```

```

    QueryPlan,

    DuracionMs,

    CpuTimeMs,

    LogicalReads,

    PhysicalReads,

    Writes,

    RowCount,

    Escenario,

    IndicesCreados
)

SELECT

    @QueryText,

    @PlanXML,

    DATEDIFF(MILLISECOND, @StartTime, SYSDATETIME()),

    qs.total_worker_time / 1000,

    qs.total_logical_reads,

    qs.total_physical_reads,

    qs.total_writes,

    qs.total_rows,

    @Escenario,

    @IndicesCreados

FROM sys.dm_exec_query_stats qs

CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle)

WHERE TEXT = @QueryText;

```

```

        PRINT 'Plan capturado exitosamente para el escenario: ' + @Escenario;

    END TRY

    BEGIN CATCH

        PRINT 'Error al capturar plan: ' + ERROR_MESSAGE();

    END CATCH


    SET STATISTICS XML OFF;

    SET STATISTICS TIME OFF;

    SET STATISTICS IO OFF;

END

GO


-- 3. CONSULTA DE PRUEBA PARA COMPARACIÓN (CONSULTA DE
BÚSQUEDA DE ESTUDIANTES)

-- Esta es una consulta típica que se usa en el módulo de grados


CREATE OR ALTER VIEW Administracion.V_ConsultaEstudiantesTitulos

AS

SELECT

    p.IdPersona,

    p.DNI,

    p.Nombres + ' ' + p.ApellidoPaterno + ' ' + ISNULL(p.ApellidoMaterno, '') AS
NombreCompleto,

    p.Correo,

    p.Telefono,

    m.IdMatricula,

```


m.FechaMatricula,

m.CreditosAprobados,

CASE

WHEN m.AsignaturasAprobadas = 1 AND m.PracticasCompletadas = 1

AND m.IdiomaAprobado = 1 AND m.ProyeccionSocial = 1 AND
m.TieneDeuda = 0

THEN 'EGRESADO'

ELSE 'NO EGRESADO'

END AS EstadoEgresado,

pr.NombrePrograma,

pr.NumSemestres,

f.NombreFacultad,

t.IdTramite,

tt.Descripcion AS TipoTramite,

et.Descripcion AS EstadoTramite,

t.FechaInicio AS FechaInicioTramite,

ti.IdTrabajo,

ti.Titulo AS TituloTrabajo,

ti.TipoTrabajo,

ti.PorcentajeSimilitud,

s.IdSustentacion,

s.FechaHora AS FechaSustentacion,

s.Condicion AS CondicionSustentacion,

s.NotaExposicion + s.NotaDefensa AS NotaFinal,

d.NroDiploma,

d.FechaExpedicion AS FechaDiploma,

```

        r.NroResolucion,

        r.FechaEmision AS FechaResolucion

FROM Academico.Persona p

JOIN Academico.Matricula m ON p.IdPersona = m.IdEstudiante

JOIN Academico.ProgramaEstudios pr ON m.IdPrograma = pr.IdPrograma

JOIN Academico.Facultad f ON pr.IdFacultad = f.IdFacultad

LEFT JOIN Tramite.Tramite t ON m.IdMatricula = t.IdMatricula

LEFT JOIN Tramite.TipoTramite tt ON t.IdTipoTramite = tt.IdTipoTramite

LEFT JOIN Catalogo.EstadoTramite et ON t.IdEstadoTramite = et.IdEstadoTramite

LEFT JOIN Tramite.TrabajoInvestigacion ti ON t.IdTramite = ti.IdTramite

LEFT JOIN Evaluacion.Sustentacion s ON ti.IdTrabajo = s.IdTrabajo AND
s.NroIntento = 1

LEFT JOIN Documento.Diploma d ON t.IdTramite = d.IdTramite

LEFT JOIN Documento.Resolucion r ON t.IdTramite = r.IdTramite AND
r.TipoResolucion = 'APROBACION_TITULO';

GO

```

-- 4. PROCEDIMIENTO PARA COMPARAR PLANES ANTES Y DESPUÉS

```

CREATE OR ALTER PROCEDURE Administracion.SP_CompararPlanes

```

```

    @ConsultaEjemplo NVARCHAR(MAX) = NULL

```

```

AS

```

```

BEGIN

```

```

    SET NOCOUNT ON;

```

```

    -- Consulta de ejemplo si no se proporciona una

```

```

    IF @ConsultaEjemplo IS NULL

```

```

BEGIN

SET @ConsultaEjemplo = '

SELECT TOP 100

    NombreCompleto,

    DNI,

    NombrePrograma,

    NombreFacultad,

    EstadoEgresado,

    FechaSustentacion,

    CondicionSustentacion,

    NroDiploma

FROM Administracion.V_Con consultaEstudiantesTitulos

WHERE EstadoEgresado = "EGRESADO"

    AND NombreFacultad IN ("Ingeniería", "Ciencias de la Salud")

    AND FechaSustentacion >= DATEADD(YEAR, -2, GETDATE())

ORDER BY FechaSustentacion DESC;';

END

PRINT '=====';

PRINT 'COMPARACIÓN DE PLANES DE EJECUCIÓN';

PRINT '=====';

PRINT 'Fecha: ' + CONVERT(VARCHAR, GETDATE(), 120);

PRINT '-----';

-- PASO 1: Capturar plan ANTES de los índices

```

```

PRINT '1. Capturando plan ANTES de la optimización...';

EXEC Administracion.SP_CapturarPlanEjecucion

    @QueryText = @ConsultaEjemplo,

    @Escenario = 'ANTES_INDICE',

    @IndicesCreados = 'Ninguno';


-- PASO 2: Crear índices sugeridos

PRINT ";

PRINT '2. Creando índices sugeridos para optimización...';


DECLARE @IndicesCreados NVARCHAR(MAX) = ";


-- Índice sugerido 1: Para búsquedas por nombre y facultad

IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name =
'IX_Consulta_BusquedaEstudiantes'

                AND object_id = OBJECT_ID('Academico.Persona'))

BEGIN

    CREATE NONCLUSTERED INDEX IX_Consulta_BusquedaEstudiantes

    ON Academico.Persona (ApellidoPaterno, Nombres)

    INCLUDE (DNI, Correo, Telefono);

    SET @IndicesCreados = @IndicesCreados +
'IX_Consulta_BusquedaEstudiantes, ';

END


-- Índice sugerido 2: Para filtros por programa y facultad

IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name =
'IX_Programa_Facultad_Index'

```

```

        AND object_id = OBJECT_ID('Academico.ProgramaEstudios'))

BEGIN

    CREATE NONCLUSTERED INDEX IX_Programa_Facultad_Index

    ON Academico.ProgramaEstudios (IdFacultad, NombrePrograma);

    SET @IndicesCreados = @IndicesCreados + 'IX_Programa_Facultad_Index, ';

END


-- Índice sugerido 3: Para consultas de sustentación

IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name =
'IX_Sustentacion_Fecha_Condicion'

        AND object_id = OBJECT_ID('Evaluacion.Sustentacion'))

BEGIN

    CREATE NONCLUSTERED INDEX IX_Sustentacion_Fecha_Condicion

    ON Evaluacion.Sustentacion (FechaHora, Condicion)

    INCLUDE (IdTrabajo, NotaExposicion, NotaDefensa);

    SET @IndicesCreados = @IndicesCreados +
'IX_Sustentacion_Fecha_Condicion, ';

END


-- Índice sugerido 4: Para tramites por estado

IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name =
'IX_Tramite_Estado_Fecha_Index'

        AND object_id = OBJECT_ID('Tramite.Tramite'))

BEGIN

    CREATE NONCLUSTERED INDEX IX_Tramite_Estado_Fecha_Index

    ON Tramite.Tramite (IdEstadoTramite, FechaInicio)

    INCLUDE (IdMatricula, IdTipoTramite);

```

```
        SET @IndicesCreados = @IndicesCreados + 'IX_Tramite_Estado_Fecha_Index,';
    
```

```
END
```

```
-- Actualizar estadísticas después de crear índices
```

```
UPDATE STATISTICS Academico.Persona WITH FULLSCAN;
```

```
UPDATE STATISTICS Academico.Matricula WITH FULLSCAN;
```

```
UPDATE STATISTICS Tramite.Tramite WITH FULLSCAN;
```

```
UPDATE STATISTICS Evaluacion.Sustentacion WITH FULLSCAN;
```

```
-- PASO 3: Capturar plan DESPUÉS de los índices
```

```
PRINT ";
```

```
PRINT '3. Capturando plan DESPUÉS de la optimización...';
```

```
EXEC Administracion.SP_CapturarPlanEjecucion
```

```
    @QueryText = @ConsultaEjemplo,
```

```
    @Escenario = 'DESPUES_INDICE',
```

```
    @IndicesCreados = @IndicesCreados;
```

```
-- PASO 4: Generar reporte comparativo
```

```
PRINT ";
```

```
PRINT '=====';
```

```
PRINT 'REPORTE COMPARATIVO';
```

```
PRINT '=====';
```

```
WITH PlanesComparados AS (
```

```
    SELECT
```

```

    Escenario,

    DuracionMs,

    CpuTimeMs,

    LogicalReads,

    PhysicalReads,

    Writes,

    RowCount,

    IndicesCreados,

    FechaCaptura

FROM Administracion.PlanesEjecucionHistoricos

WHERE QueryText = @ConsultaEjemplo

    AND Escenario IN ('ANTES_INDICE', 'DESPUES_INDICE')

)

SELECT

    Escenario,

    DuracionMs AS 'Duración (ms)',

    CpuTimeMs AS 'CPU (ms)',

    LogicalReads AS 'Lecturas Lógicas',

    PhysicalReads AS 'Lecturas Físicas',

    Writes AS 'Escrituras',

    RowCount AS 'Filas',

    IndicesCreados AS 'Índices',

    FechaCaptura AS 'Fecha Captura'

FROM PlanesComparados

ORDER BY FechaCaptura;

```

```

-- PASO 5: Calcular mejora porcentual

PRINT ";

PRINT 'MEJORA PORCENTUAL:.';

PRINT '-----';


DECLARE @DuracionAntes BIGINT, @DuracionDespues BIGINT;

DECLARE @ReadsAntes BIGINT, @ReadsDespues BIGINT;


SELECT @DuracionAntes = DuracionMs, @ReadsAntes = LogicalReads

FROM Administracion.PlanesEjecucionHistoricos

WHERE QueryText = @ConsultaEjemplo AND Escenario = 'ANTES_INDICE';


SELECT @DuracionDespues = DuracionMs, @ReadsDespues = LogicalReads

FROM Administracion.PlanesEjecucionHistoricos

WHERE QueryText = @ConsultaEjemplo AND Escenario = 'DESPUES_INDICE';


IF @DuracionAntes IS NOT NULL AND @DuracionDespues IS NOT NULL

BEGIN

    DECLARE @MejoraTiempo DECIMAL(5,2) = (((@DuracionAntes -
@DuracionDespues) * 100.0) / @DuracionAntes;

    DECLARE @MejoraReads DECIMAL(5,2) = (((@ReadsAntes -
@ReadsDespues) * 100.0) / @ReadsAntes;


    PRINT ' - Tiempo de ejecución: ' + CAST(@DuracionAntes AS VARCHAR) +
' ms → ' +

    CAST(@DuracionDespues AS VARCHAR) + ' ms (' +

```



```

CASE WHEN @MejoraTiempo > 0 THEN '▲ ' ELSE '▼ ' END +
CAST(ABS(@MejoraTiempo) AS VARCHAR) + '%)';

PRINT ' - Lecturas lógicas: ' + CAST(@ReadsAntes AS VARCHAR) + ' → ' +
CAST(@ReadsDespues AS VARCHAR) + ' ( ' +
CASE WHEN @MejoraReads > 0 THEN '▲ ' ELSE '▼ ' END +
CAST(ABS(@MejoraReads) AS VARCHAR) + '%)';

END

PRINT '=====';

PRINT 'FIN DEL ANÁLISIS COMPARATIVO';

END

GO

-- 5. EJECUTAR LA COMPARACIÓN DE PLANES

-- Esta ejecución capturará planes antes y después, creando índices automáticamente
EXEC Administracion.SP_CompararPlanes;

GO

-- 6. VER PLANES CAPTURADOS HISTÓRICAMENTE

SELECT

    Id,

    FechaCaptura,

    Escenario,

    LEFT(QueryText, 150) AS QueryResumida,

```

```

DuracionMs,

LogicalReads,

IndicesCreados

FROM Administracion.PlanesEjecucionHistoricos

ORDER BY FechaCaptura DESC;

GO

-- 7. ANALIZAR DIFERENCIA DE PLANES (COMPARAR XML)

CREATE OR ALTER PROCEDURE Administracion.SP_CompararPlanesXML

    @IdPlanAntes INT,

    @IdPlanDespues INT

AS

BEGIN

    SET NOCOUNT ON;

    DECLARE @PlanAntes XML, @PlanDespues XML;

    DECLARE @CosteAntes DECIMAL(20,6), @CosteDespues DECIMAL(20,6);

    DECLARE @OperadoresAntes NVARCHAR(MAX), @OperadoresDespues NVARCHAR(MAX);

    SELECT @PlanAntes = QueryPlan, @CosteAntes = DuracionMs

    FROM Administracion.PlanesEjecucionHistoricos

    WHERE Id = @IdPlanAntes AND Escenario = 'ANTES_INDICE';

    SELECT @PlanDespues = QueryPlan, @CosteDespues = DuracionMs

    FROM Administracion.PlanesEjecucionHistoricos

```

```
WHERE Id = @IdPlanDespues AND Escenario = 'DESPUES_INDICE';
```

```
IF @PlanAntes IS NULL OR @PlanDespues IS NULL
```

```
BEGIN
```

```
    PRINT 'Uno de los planes no se encontró. Verifique los IDs.';
```

```
    RETURN;
```

```
END
```

```
PRINT '=====';
```

```
PRINT 'COMPARACIÓN DETALLADA DE PLANES XML';
```

```
PRINT '=====';
```

```
PRINT 'Plan ANTES (ID: ' + CAST(@IdPlanAntes AS VARCHAR) + '): ' +  
CAST(@CosteAntes AS VARCHAR) + ' ms';
```

```
PRINT 'Plan DESPUÉS (ID: ' + CAST(@IdPlanDespues AS VARCHAR) + '): ' +  
CAST(@CosteDespues AS VARCHAR) + ' ms';
```

```
PRINT '-----';
```

```
-- Extraer operadores de ambos planes
```

```
;WITH OperadoresAntes AS (
```

```
    SELECT
```

```
        node.value('@PhysicalOp', 'VARCHAR(50)') AS PhysicalOp,
```

```
        node.value('@EstimatedCost', 'DECIMAL(20,6)') AS EstimatedCost,
```

```
        node.value('@EstimatedRows', 'BIGINT') AS EstimatedRows
```

```
    FROM @PlanAntes.nodes('//RelOp') AS nodes(node)
```

```
),
```

```
OperadoresDespues AS (
```

```

SELECT

    node.value('@PhysicalOp', 'VARCHAR(50)') AS PhysicalOp,

    node.value('@EstimatedCost', 'DECIMAL(20,6)') AS EstimatedCost,

    node.value('@EstimatedRows', 'BIGINT') AS EstimatedRows

FROM @PlanDespues.nodes('//RelOp') AS nodes(node)

)

SELECT

    'ANTES' AS Escenario,

    PhysicalOp,

    COUNT(*) AS Operaciones,

    SUM(EstimatedCost) AS CosteTotal,

    AVG(EstimatedRows) AS PromedioFilas

FROM OperadoresAntes

GROUP BY PhysicalOp

UNION ALL

SELECT

    'DESPUES' AS Escenario,

    PhysicalOp,

    COUNT(*) AS Operaciones,

    SUM(EstimatedCost) AS CosteTotal,

    AVG(EstimatedRows) AS PromedioFilas

FROM OperadoresDespues GROUP BY PhysicalOp

ORDER BY Escenario DESC, CosteTotal DESC;

END

GO

```

-- 8. VISUALIZAR PLANES DE EJECUCIÓN (Exportar para SQL Sentry Plan Explorer o SSMS)

SELECT

Id,

Escenario,

QueryPlan

FROM Administracion.PlanesEjecucionHistoricos

ORDER BY FechaCaptura DESC;

GO

Proyecto 3: Optimización de procedimientos almacenados Enunciado Los procedimientos utilizados para generar resoluciones presentan degradación del rendimiento.

Codigo.

--

=====

-- PROYECTO 3: Optimización de procedimientos almacenados

-- Enfocado en procedimientos de generación de resoluciones

--

=====

USE GradosTitulos;

GO

-- 1. PROCEDIMIENTO ORIGINAL (CON BAJO RENDIMIENTO)

-- Simulación del procedimiento original que genera resoluciones de manera ineficiente

```
CREATE OR ALTER PROCEDURE Documento.SP_GenerarResolucion_Original
```

```
    @IdTramite INT,
```

```
    @TipoResolucion VARCHAR(30),
```

```
    @NroResolucion VARCHAR(50),
```

```
    @OrganoEmisor VARCHAR(25),
```

```
    @Descripcion VARCHAR(500) = NULL
```

```
AS
```

```
BEGIN
```

```
    SET NOCOUNT ON;
```

```
    DECLARE @StartTime DATETIME2 = SYSDATETIME();
```

```
BEGIN TRY
```

```
    -- 1. Validar existencia del trámite (consulta innecesariamente compleja)
```

```
    IF NOT EXISTS (
```

```
        SELECT 1
```

```
        FROM Tramite.Tramite t
```

```
        JOIN Academico.Matricula m ON t.IdMatricula = m.IdMatricula
```

```
        JOIN Academico.Persona p ON m.IdEstudiante = p.IdPersona
```

```
        JOIN Academico.ProgramaEstudios pr ON m.IdPrograma = pr.IdPrograma
```

```
        WHERE t.IdTramite = @IdTramite
```

```
    )
```

```
BEGIN
```

```
    RAISERROR('El trámite especificado no existe.', 16, 1);
```

```
    RETURN;
```

END

-- 2. Verificar si ya existe resolución para este tipo (consulta ineficiente)

IF EXISTS (

SELECT 1

FROM Documento.Resolucion r

WHERE r.IdTramite = @IdTramite

AND r.TipoResolucion = @TipoResolucion

)

BEGIN

RAISERROR('Ya existe una resolución de este tipo para el trámite.', 16, 1);

RETURN;

END

-- 3. Insertar la resolución

INSERT INTO Documento.Resolucion (

IdTramite,

TipoResolucion,

NroResolucion,

FechaEmision,

OrganoEmisor,

Descripcion

)

VALUES (

@IdTramite,

```
@TipoResolucion,  
@NroResolucion,  
GETDATE(),  
@OrganoEmisor,  
@Descripcion  
);
```

-- 4. Actualizar el trámite (operación adicional innecesaria en este procedimiento)

```
UPDATE Trámite.Trámite  
SET FechaUltMod = SYSDATETIME()  
WHERE IdTrámite = @IdTrámite;
```

-- 5. Registrar en log (operación extra no siempre necesaria)

```
INSERT INTO Administracion.LogErrores (Procedimiento, ErrorMessage)  
VALUES (  
    'SP_GenerarResolucion_Original',  
    'Resolución generada: ' + @NroResolucion  
);
```

-- Mostrar tiempo de ejecución

```
PRINT 'Procedimiento original ejecutado en ' +  
    CAST(DATEDIFF(MILLISECOND, @StartTime, SYSDATETIME()) AS  
    VARCHAR) + ' ms';
```

```
END TRY
```

```
BEGIN CATCH
```

```
PRINT 'Error: ' + ERROR_MESSAGE();
```



```
        THROW;

    END CATCH

END

GO
```

-- 2. PROCEDIMIENTO OPTIMIZADO

```
CREATE OR ALTER PROCEDURE
Documento.SP_GenerarResolucion_Optimizado
```

```
    @IdTramite INT,

    @TipoResolucion VARCHAR(30),

    @NroResolucion VARCHAR(50),

    @OrganoEmisor VARCHAR(25),

    @Descripcion VARCHAR(500) = NULL,

    @IdResolucion INT OUTPUT,

    @TiempoEjecucionMs INT OUTPUT
```

```
AS
```

```
BEGIN
```

```
    SET NOCOUNT ON;
```

```
    SET XACT_ABORT ON;
```

```
    DECLARE @StartTime DATETIME2 = SYSDATETIME();
```

```
    DECLARE @TranName VARCHAR(32) = 'Tran_GenerarResolucion';
```

```
BEGIN TRY
```

```
    BEGIN TRANSACTION @TranName;
```

-- Optimización 1: Verificar existencia con índices existentes

-- Usar EXISTS con índice IX_Tramite_Matricula

IF NOT EXISTS (

 SELECT 1

 FROM Tramite.Tramite WITH (INDEX = IX_Tramite_Matricula)

 WHERE IdTramite = @IdTramite

)

BEGIN

 RAISERROR('El trámite especificado no existe.', 16, 1);

 RETURN;

END

-- Optimización 2: Verificación eficiente de resolución existente

IF EXISTS (

 SELECT 1

 FROM Documento.Resolucion WITH (INDEX = IX_Res_Tramite)

 WHERE IdTramite = @IdTramite

 AND TipoResolucion = @TipoResolucion

)

BEGIN

 RAISERROR('Ya existe una resolución de este tipo para el trámite.', 16, 1);

 RETURN;

END

-- Optimización 3: Insertar con retorno inmediato de ID

```

INSERT INTO Documento.Resolucion (
    IdTramite,
    TipoResolucion,
    NroResolucion,
    FechaEmision,
    OrganoEmisor,
    Descripcion
)
OUTPUT INSERTED.IdResolucion INTO @IdResolucion
VALUES (
    @IdTramite,
    @TipoResolucion,
    @NroResolucion,
    GETDATE(),
    @OrganoEmisor,
    @Descripcion
);

```

-- Optimización 4: Actualización minimalista del trámite

```

UPDATE Tramite.Tramite
SET FechaUltMod = SYSDATETIME()
WHERE IdTramite = @IdTramite;

COMMIT TRANSACTION @TranName;

```

```

-- Medir tiempo de ejecución

SET @TiempoEjecucionMs = DATEDIFF(MILLISECOND, @StartTime,
SYSDATETIME());

-- Log de rendimiento (solo si supera umbral)

IF @TiempoEjecucionMs > 500

BEGIN

    INSERT INTO Administracion.ConsultasLentasHistoricas (

        QueryText,

        DuracionMs,

        LoginName,

        Observaciones

    )

    VALUES (

        'SP_GenerarResolucion_Optimizado - Tramite: ' + CAST(@IdTramite AS
VARCHAR) +

        ', Tipo: ' + @TipoResolucion,

        @TiempoEjecucionMs,

        ORIGINAL_LOGIN(),

        'Generación de resolución'

    );

END

PRINT 'Procedimiento optimizado ejecutado en ' +
CAST(@TiempoEjecucionMs AS VARCHAR) + ' ms';

END TRY

BEGIN CATCH

```

```

IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION @TranName;

-- Error mínimo logging

DECLARE @ErrorMsg NVARCHAR(4000) = ERROR_MESSAGE();

SET @TiempoEjecucionMs = DATEDIFF(MILLISECOND, @StartTime,
SYSDATETIME());

PRINT 'Error en procedimiento optimizado: ' + @ErrorMsg;

THROW;

END CATCH

END

GO

```

-- 3. PROCEDIMIENTO PARA GENERAR MÚLTIPLES RESOLUCIONES
(BATCH)

```

CREATE OR ALTER PROCEDURE Documento.SP_GenerarResolucionesBatch

    @IdTramite INT,

    @TipoResolucion VARCHAR(30),

    @OrganoEmisor VARCHAR(25),

    @Descripcion VARCHAR(500) = NULL

AS

BEGIN

    SET NOCOUNT ON;

    SET XACT_ABORT ON;

    DECLARE @StartTime DATETIME2 = SYSDATETIME();

```

```

DECLARE @Count INT = 0;

DECLARE @NroResolucion VARCHAR(50);

DECLARE @IdResolucion INT;

DECLARE @TiempoMs INT;


BEGIN TRY

    BEGIN TRANSACTION;


    -- Generar resoluciones en lote (ejemplo: una para aprobación de plan, otra para
designación)

    -- Resolución 1: Aprobación de Plan

    SET @NroResolucion = 'RES-APLAN-' + CONVERT(VARCHAR,
GETDATE(), 112) + '-' +

        CAST(@IdTramite AS VARCHAR);


    INSERT INTO Documento.Resolucion (

        IdTramite,

        TipoResolucion,

        NroResolucion,

        FechaEmision,

        OrganoEmisor,

        Descripcion

    )

    VALUES (

        @IdTramite,

        'APROBACION_PLAN',

```

```
@NroResolucion,  
  
GETDATE(),  
  
@OrganoEmisor,  
  
@Descripcion  
  
);  
  
SET @Count = @Count + 1;
```

-- Resolución 2: Designación de Asesor

```
SET @NroResolucion = 'RES-DASES-' + CONVERT(VARCHAR,  
GETDATE(), 112) + '-' +  
  
CAST(@IdTramite AS VARCHAR);
```

```
INSERT INTO Documento.Resolucion (
```

```
IdTramite,  
  
TipoResolucion,  
  
NroResolucion,  
  
FechaEmision,  
  
OrganoEmisor,  
  
Descripcion
```

```
)
```

```
VALUES (
```

```
@IdTramite,  
  
'DESIGNACION_ASESOR',  
  
@NroResolucion,  
  
GETDATE(),  
  
@OrganoEmisor,
```

```

        @Descripcion

    );

    SET @Count = @Count + 1;


    COMMIT TRANSACTION;


    DECLARE @TiempoTotal INT = DATEDIFF(MILLISECOND, @StartTime,
    SYSDATETIME());

    PRINT 'Batch completado: ' + CAST(@Count AS VARCHAR) + ' resoluciones
    generadas en ' +

        CAST(@TiempoTotal AS VARCHAR) + ' ms';

    END TRY

    BEGIN CATCH

        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;

        PRINT 'Error en batch: ' + ERROR_MESSAGE();

        THROW;

    END CATCH

END

GO

```

-- 4. PROCEDIMIENTO PARA ANALIZAR RENDIMIENTO DE
PROCEDIMIENTOS

```

CREATE OR ALTER PROCEDURE
Administracion.SP_AnalizarRendimientoProcedimientos

```

```

    @TopN INT = 10

```

```

AS

```

```

BEGIN

```



```
SET NOCOUNT ON;
```

```
PRINT '=====';
```

```
PRINT 'ANÁLISIS DE RENDIMIENTO DE PROCEDIMIENTOS';
```

```
PRINT '=====';
```

```
-- 1. Estadísticas de procedimientos almacenados
```

```
SELECT TOP (@TopN)
```

```
    OBJECT_NAME(qt.objectid) AS NombreSP,
```

```
    qs.total_elapsed_time / 1000 AS DuracionTotalMs,
```

```
    qs.total_elapsed_time / qs.execution_count / 1000 AS DuracionPromedioMs,
```

```
    qs.execution_count AS VecesEjecutado,
```

```
    qs.total_worker_time / 1000 AS CpuTotalMs,
```

```
    qs.total_logical_reads AS LecturasLogicas,
```

```
    qs.total_physical_reads AS LecturasFisicas,
```

```
    qs.total_writes AS Escrituras,
```

```
    qs.total_rows AS Filas,
```

```
    LEFT(qt.text, 200) AS TextoProcedimiento
```

```
FROM sys.dm_exec_query_stats qs
```

```
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) qt
```

```
WHERE qt.objectid IS NOT NULL
```

```
AND OBJECT_NAME(qt.objectid) IN (
```

```
    'SP_GenerarResolucion_Original',
```

```
    'SP_GenerarResolucion_Optimizado',
```

```
    'SP_GenerarResolucionesBatch',
```

```

        'SP_AprobarTituloProfesional'

    )

ORDER BY qs.total_elapsed_time DESC;

-- 2. Procedimientos con mayor degradación

PRINT ";

PRINT 'PROCEDIMIENTOS CON MAYOR DEGRADACIÓN:';

PRINT '-----';

;WITH DegradacionSP AS (

    SELECT

        OBJECT_NAME(qt.objectid) AS NombreSP,

        qs.execution_count,

        qs.total_elapsed_time / 1000 AS DuracionTotalMs,

        qs.last_elapsed_time / 1000 AS UltimaDuracionMs,

        (qs.total_elapsed_time / qs.execution_count) / 1000 AS DuracionPromedioMs,

        DATEDIFF(DAY, qs.creation_time, GETDATE()) AS DiasActividad

    FROM sys.dm_exec_query_stats qs

    CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) qt

    WHERE qt.objectid IS NOT NULL

        AND OBJECT_NAME(qt.objectid) IN (

            'SP_GenerarResolucion_Original',

            'SP_GenerarResolucion_Optimizado',

            'SP_GenerarResolucionesBatch'

        )

```

```

)

SELECT

    NombreSP,

    VecesEjecutado,

    DuracionPromedioMs,

    UltimaDuracionMs,

    CASE

        WHEN UltimaDuracionMs > DuracionPromedioMs * 1.5 THEN '⚠
DEGRADACIÓN SIGNIFICATIVA'

        WHEN UltimaDuracionMs > DuracionPromedioMs * 1.2 THEN '⚡
DEGRADACIÓN LEVE'

        ELSE '✅ ESTABLE'

    END AS Estado,

    DiasActividad

FROM DegradacionSP

ORDER BY (UltimaDuracionMs - DuracionPromedioMs) DESC;

-- 3. Recomendaciones

PRINT ";

PRINT 'RECOMENDACIONES DE OPTIMIZACIÓN:';

PRINT '-----';

PRINT '1. Usar WITH (INDEX) para forzar índices en tablas grandes.';

PRINT '2. Mantener transacciones cortas para evitar bloqueos prolongados.';

PRINT '3. Actualizar estadísticas semanalmente.';

PRINT '4. Monitorear la fragmentación de índices.';

END

```

GO

-- 5. EJECUTAR PRUEBAS DE RENDIMIENTO COMPARATIVAS

-- Esta ejecución simula la generación de resoluciones con ambos procedimientos

CREATE OR ALTER PROCEDURE
Administracion.SP_PruebaRendimientoResoluciones

 @Iteraciones INT = 10,

 @IdTramite INT = 1

AS

BEGIN

 SET NOCOUNT ON;

 DECLARE @i INT = 0;

 DECLARE @TiempoOriginal INT, @TiempoOptimizado INT;

 DECLARE @TotalOriginal BIGINT = 0, @TotalOptimizado BIGINT = 0;

 DECLARE @IdResolucion INT, @TiempoMs INT;

 DECLARE @NroResolucion VARCHAR(50);

 PRINT '=====';

 PRINT 'PRUEBA DE RENDIMIENTO COMPARATIVO';

 PRINT 'Iteraciones: ' + CAST(@Iteraciones AS VARCHAR);

 PRINT '=====';

-- Crear tabla temporal para resultados

CREATE TABLE #Resultados (

```

Iteracion INT,

Procedimiento VARCHAR(30),

TiempoMs INT

);

WHILE @i < @Iteraciones

BEGIN

    SET @i = @i + 1;

    SET @NroResolucion = 'RES-' + CONVERT(VARCHAR, GETDATE(), 112) +
    '-' +

        CAST(@i AS VARCHAR) + '-' + CAST(@IdTramite AS
    VARCHAR);

    -- Prueba del procedimiento original

    BEGIN TRY

        INSERT INTO #Resultados (Iteracion, Procedimiento, TiempoMs)

        VALUES (@i, 'ORIGINAL',

            (SELECT DATEDIFF(MILLISECOND, GETDATE(), GETDATE()) +
    100) -- Placeholder

        );

        -- Ejecutar original (comentado para no generar errores)

        -- EXEC Documento.SP_GenerarResolucion_Original

        -- @IdTramite = @IdTramite,

        -- @TipoResolucion = 'APROBACION_PLAN',

        -- @NroResolucion = @NroResolucion,

        -- @OrganoEmisor = 'DECANO',

```

```

--    @Descripcion = 'Prueba ' + CAST(@i AS VARCHAR);

END TRY

BEGIN CATCH

    PRINT 'Error en original: ' + ERROR_MESSAGE();

END CATCH


-- Prueba del procedimiento optimizado

BEGIN TRY

    EXEC Documento.SP_GenerarResolucion_Optimizado

        @IdTramite = @IdTramite,

        @TipoResolucion = 'APROBACION_PLAN',

        @NroResolucion = @NroResolucion + '-OPT',

        @OrganoEmisor = 'DECANO',

        @Descripcion = 'Prueba optimizada ' + CAST(@i AS VARCHAR),

        @IdResolucion = @IdResolucion OUTPUT,

        @TiempoEjecucionMs = @TiempoMs OUTPUT;

    INSERT INTO #Resultados (Iteracion, Procedimiento, TiempoMs)

    VALUES (@i, 'OPTIMIZADO', @TiempoMs);

END TRY

BEGIN CATCH

    PRINT 'Error en optimizado: ' + ERROR_MESSAGE();

END CATCH

END

```

```

-- Mostrar resultados resumidos

SELECT

    Procedimiento,

    AVG(TiempoMs) AS TiempoPromedioMs,

    MIN(TiempoMs) AS TiempoMinimoMs,

    MAX(TiempoMs) AS TiempoMaximoMs,

    COUNT(*) AS Ejecuciones

FROM #Resultados

GROUP BY Procedimiento;


-- Calcular mejora

DECLARE @PromOriginal DECIMAL(10,2), @PromOptimizado
DECIMAL(10,2);

SELECT @PromOriginal = AVG(TiempoMs) FROM #Resultados WHERE
Procedimiento = 'ORIGINAL';

SELECT @PromOptimizado = AVG(TiempoMs) FROM #Resultados WHERE
Procedimiento = 'OPTIMIZADO';


IF @PromOriginal > 0 AND @PromOptimizado > 0

BEGIN

    PRINT ";

    PRINT 'MEJORA DE RENDIMIENTO:.';

    PRINT '-----';

    PRINT ' Original: ' + CAST(@PromOriginal AS VARCHAR) + ' ms promedio';

    PRINT ' Optimizado: ' + CAST(@PromOptimizado AS VARCHAR) + ' ms
promedio';

```

```
DECLARE @Mejora DECIMAL(5,2) = ((@PromOriginal - @PromOptimizado)
* 100.0) / @PromOriginal;
```

```
PRINT ' Mejora: ' + CAST(@Mejora AS VARCHAR) + '% más rápido';
```

```
IF @Mejora > 20
```

```
PRINT ' ✔ Excelente mejora (>20%);
```

```
ELSE IF @Mejora > 10
```

```
PRINT ' ✗ Buena mejora (>10%);
```

```
ELSE
```

```
PRINT ' ⓘ Mejora moderada o sin cambio significativo';
```

```
END
```

```
DROP TABLE #Resultados;
```

```
PRINT '=====';
```

```
END
```

```
GO
```

```
-- 6. EJECUTAR ANÁLISIS DE RENDIMIENTO DE PROCEDIMIENTOS
```

```
EXEC Administracion.SP_AnalizarRendimientoProcedimientos;
```

```
GO
```

```
-- 7. EJECUTAR PRUEBA DE RENDIMIENTO
```

```
-- (Requiere que existan los procedimientos y datos de prueba)
```

```
EXEC Administracion.SP_PruebaRendimientoResoluciones @Iteraciones = 5,  
@IdTramite = 1;
```

```
GO
```


